Guia de estudo dos diagramas

Feature Request & Product Roadmap Management System (FeatureFlow)

Engenharia de Software · UBI · 2026/27

Como cada diagrama está montado, a notação que usa, como se liga aos outros e ao código do Mini Sprint 3 — com perguntas de revisão para a apresentação.

Índice

1. O mapa: como os diagramas se encaixam **4**

O fio condutor . . . 4

Como foram desenhados . . . 5

2. Notação rápida (cábula) **6**

Diagrama de Casos de Uso (UML) . . . 6

Diagrama de Classes (UML) . . . 6

Modelo Entidade-Relação (pé-de-galinha) . . . 6

Diagrama de Estados (UML) . . . 7

Diagrama de Sequência (UML) . . . 7

Diagrama de Atividade (UML) . . . 7

Modelo C4 . . . 8

3. Casos de Uso (EDS-12) **9**

Como está montado . . . 9

«include» vs «extend» — a dúvida clássica . . . 9

Diagrama 0 — Casos de uso comuns e hierarquia de atores . . . 10

Diagrama 1 — Cliente (Sofia) . . . 12

Diagrama 2 — Gestor de Produto (Tiago) . . . 14

Diagrama 3 — Programador (Rui) . . . 16

Diagrama 4 — Gestor de Projeto (Inês) . . . 18

Dos casos de uso aos outros diagramas . . . 20

4. Arquitetura — Modelo C4 (EDS-10 e EDS-11) **21**

Nível 1 — Contexto . . . 21

Nível 2 — Contentores . . . 23

Nível 3 — Componentes do back-end . . . 25

Os ADRs que explicam o C4 . . . 27

5. Diagrama de Classes (EDS-13) **28**

Como está montado . . . 28

As relações mais importantes . . . 28

As 15 decisões de design (decora as principais) . . . 28

6. Modelo Entidade-Relação (EDS-14) **32**

Regras garantidas pela base de dados . . . 32

7. Diagramas de Estados **35**

Diagrama 1 — Pedido . . . 35

Diagrama 2 — User Story . . . 37

8. Diagramas de Sequência 1–4 **40**

Sequência 1 — Submeter e triar pedido com IA . . . 40

Sequência 2 — Aprovar pedido e gerar User Story . . . 42

Sequência 3 — Votar e retirar voto . . . 44

Sequência 4 — Mover User Story para o Sprint (Definition of Ready) . . . 46

9. Diagrama de Atividade — do Pedido à User Story **49**

Como está montado . . . 49

10. Coerência entre diagramas e código **52**

Matriz de rastreabilidade . . . 52

Diferenças que deves conhecer (e saber justificar) . . . 52

11. Revisão rápida **54**

Os números e nomes que convém saber de cor . . . 54

Exercício para treinar . . . 54

Onde está cada coisa . . . 54

1. O mapa: como os diagramas se encaixam

Os diagramas não são peças soltas: cada um responde a uma pergunta diferente sobre o mesmo sistema e todos usam **os mesmos nomes** (classes, estados, operações e endpoints). É essa coerência que o critério C2 da avaliação pede ("coerência entre requisitos, design, wireframes e implementação").

Pergunta	Diagrama	Tipo de vista	Documento
Quem faz o quê no sistema?	Casos de Uso (5 diagramas)	Funcional (requisitos)	EDS-12
Quem usa o sistema e com que sistemas externos fala?	C4 nível 1 — Contexto	Estrutura (arquitetura)	EDS-11
Que peças correm separadamente e como comunicam?	C4 nível 2 — Contentores	Estrutura (arquitetura)	EDS-10, EDS-11
Como está organizado o back-end por dentro?	C4 nível 3 — Componentes	Estrutura (arquitetura)	EDS-10, EDS-11
Que conceitos existem e como se relacionam?	Diagrama de Classes	Estrutura (domínio)	EDS-13
Como ficam guardados na base de dados?	Modelo Entidade-Relação	Estrutura (dados)	EDS-14
Por que fases passa um Pedido / uma User Story?	Diagramas de Estados	Comportamento (um objeto)	Diagramas de Estados
Que mensagens trocam os contentores num cenário?	Diagramas de Sequência 1–4	Comportamento (interação)	Diagrama1…4
Qual é o fluxo de trabalho completo, por ator?	Diagrama de Atividade	Comportamento (processo)	Diagrama_Atividade

O fio condutor

Lê-se de cima para baixo, do problema até ao código:

- **Personas e User Stories** (Sofia, Tiago, Rui, Inês) → definem *o que* o sistema tem de fazer. Cada classe, estado ou passo existe porque uma US o pede (as decisões citam sempre "US 2.1", "US 1.3"…).
- **Casos de Uso** → para cada ator, os objetivos que tem no sistema; cada caso de uso cita a US de onde vem.
- **Arquitetura (C4 + ADRs)** → decide *com que peças*: SPA React, API Django, Worker Celery, Redis, PostgreSQL.
- **Classes** → vocabulário do domínio: `Pedido`, `Voto`, `aprovar()`…
- **Modelo ER** → cada classe vira tabela; associações viram chaves estrangeiras ou tabelas de associação.
- **Estados** → o ciclo de vida do Pedido e da User Story; cada operação da classe é uma transição.
- **Sequência e Atividade** → mostram os mesmos estados e operações a acontecer num cenário real, com os contentores do C4 como participantes.
- **Código** → `models.py` segue as classes, as migrações seguem o ER, os endpoints seguem os diagramas de sequência e os testes verificam as regras.

A ideia a guardar

Uma operação como `aprovar()` aparece em **seis** sítios: é o caso de uso "Aprovar pedido" (EDS-12), é método da classe Pedido (EDS-13), é a transição Em Triagem → Aprovado com a guarda [esforço e impacto definidos] (Estados), é o passo 7 do Diagrama de Sequência 2, é a atividade "Aprovar pedido" do Diagrama de Atividade e é `Pedido.aprovar()` em `backend/pedidos/models.py`, testada em `test_aprovar_falha_sem_esforco_ou_impacto`. Se te perguntarem "como garantem a coerência?", é este o exemplo.

Como foram desenhados

Todos os diagramas foram escritos em **PlantUML** (texto que gera a imagem). O código de cada um está nas últimas páginas do respetivo PDF e pode ser colado em `planttext.com` para editar. Vantagem: os diagramas ficam versionados no Git como o código e são fáceis de alterar.

2. Notação rápida (cábula)

Diagrama de Casos de Uso (UML)

Símbolo	Significado	Exemplo no projeto
Boneco (ator)	Quem interage com o sistema: pessoa ou sistema externo («sistema»)	Cliente; Serviço de IA; GitHub
Elipse	Caso de uso: um objetivo do ator	Submeter pedido (US 2.1)
Retângulo	Fronteira do sistema	Feature Request & Product Roadmap Management System
Linha simples	Associação: o ator participa no caso de uso	Cliente — Votar em pedido
«include» (seta tracejada da base para o incluído)	O caso de uso base faz **sempre** o incluído	Aprovar pedido → Avaliar esforço e impacto
«extend» (seta tracejada da extensão para a base)	Comportamento **opcional** que estende a base	Retirar voto → Votar em pedido
Seta com triângulo vazio	Generalização: o filho herda os casos de uso do pai	Cliente □ Utilizador

Diagrama de Classes (UML)

Símbolo	Significado	Exemplo no projeto
`-atributo: Tipo`	Atributo privado (- privado, + público)	`-titulo: String` em Pedido
`+operacao(): Tipo`	Operação pública (só as regras de negócio)	`+aprovar(): void`
Linha simples	Associação	Utilizador — submete — Pedido
Losango preto □	Composição: a parte não existe sem o todo (apaga em cascata)	Pedido □— Anexo; UserStory □— CriterioAceitacao
1, 0..1, *, 0..*	Multiplicidade: quantos objetos de cada lado	Pedido 0..1 — * UserStory
«enumeration»	Conjunto fechado de valores	Perfil, EstadoPedido, EstadoUserStory
Nota (folha dobrada)	Restrição que não cabe na notação	"Um utilizador só pode votar uma vez em cada pedido"

Modelo Entidade-Relação (pé-de-galinha)

Símbolo	Significado
«PK» / «FK»	Chave primária / chave estrangeira
• antes da coluna	Coluna obrigatória (NOT NULL)
|| (dois traços)	Exatamente um
o| (círculo e traço)	Zero ou um (opcional)
o{ (círculo e pé-de-galinha)	Zero ou mais

Símbolo	Significado
|{ (traço e pé-de-galinha)	Um ou mais

Diagrama de Estados (UML)

Símbolo	Significado
● círculo preto	Estado inicial
□ círculo com contorno	Estado final (o objeto termina o ciclo de vida)
Retângulo arredondado	Estado (ex.: Submetido)
`evento [guarda] / ação`	Transição: o evento dispara-a, a guarda tem de ser verdadeira, a ação executa-se
`entry / ação`	Ação feita ao entrar no estado (ex.: guardar dataSubmissao)
`do / atividade`	Atividade contínua enquanto está no estado (ex.: verificar dependências)

Diagrama de Sequência (UML)

Símbolo	Significado
Boneco / caixa no topo	Ator (pessoa) / participante (contentor); o cilindro é a base de dados, o tubo é a fila
Linha vertical tracejada	Linha de vida (o tempo corre para baixo)
Retângulo fino na linha de vida	Ativação: o participante está a processar
Seta cheia →	Mensagem (pedido / chamada)
Seta tracejada □	Resposta (return)
Seta para o próprio participante	Chamada interna (ex.: `pedido.aprovar()`)
Números 1, 2, 3…	`autonumber`: ordem das mensagens (a descrição refere "passos 1 a 9")
alt [condição]	Alternativas (if / else): só um ramo acontece
opt [condição]	Opcional (if sem else)
loop [condição]	Repetição (ex.: até 2 tentativas à IA)
== Título ==	Separador de fases (Submissão, Processamento assíncrono…)

Diagrama de Atividade (UML)

Símbolo	Significado
Raias (swimlanes)	Uma coluna por ator ou sistema: mostra *quem* faz cada atividade
Retângulo arredondado	Atividade (ação)
Losango	Decisão (if) ou junção de ramos (merge)
Barra preta grossa	Fork / join: atividades em paralelo
● / □	Início / fim do fluxo

Modelo C4

Nível	Mostra	Pergunta
1 — Contexto	O sistema como uma caixa, pessoas e sistemas externos	Quem usa e com quem fala?
2 — Contentores	Aplicações/processos que correm à parte e a tecnologia de cada um	Que peças existem e como comunicam?
3 — Componentes	O interior de um contentor (aqui, os módulos da API)	Como está organizado por dentro?
4 — Código	Classes (não usado: o diagrama de classes faz esse papel)	—

Cores do C4: azul-escuro = pessoa; azul = contentor/componente do sistema; cinzento = sistema externo; a caixa tracejada é a fronteira do sistema. Cada seta tem o que faz e o protocolo (ex.: "Chama a API [REST/JSON, HTTPS]").

3. Casos de Uso (EDS-12)

É o diagrama mais próximo dos requisitos: mostra **quem** (ator) quer fazer **o quê** (caso de uso) no sistema, sem dizer como. Vem logo a seguir às Personas e User Stories: entre parênteses, cada caso de uso indica a US que lhe deu origem (ex.: "Votar em pedido (US 2.2)"). Os diagramas de sequência depois mostram *como* cada caso de uso acontece.

Como está montado

Em vez de um diagrama gigante, há **cinco**: um com o que é comum a todos e um por persona.

Diagrama	Ator principal	Atores secundários	N.º de casos de uso
Comuns e hierarquia de atores	Utilizador (e os 4 perfis por generalização)	—	6
Cliente	Cliente (Sofia)	—	9
Gestor de Produto	Gestor de Produto (Tiago)	Serviço de IA «sistema»	19
Programador	Programador (Rui)	GitHub «sistema»	8
Gestor de Projeto	Gestor de Projeto (Inês)	—	9

- **Ator principal** (à esquerda): quem tem o objetivo. **Ator secundário** (à direita, «sistema»): sistema externo que o sistema usa para cumprir o caso de uso — os mesmos sistemas externos do C4 nível 1.
- **Generalização:** Cliente, Gestor de Produto, Programador e Gestor de Projeto são todos Utilizadores, por isso herdam os casos de uso comuns (autenticar-se, comentar, notificações…). Bate certo com o atributo `perfil` do Utilizador no diagrama de classes (decisão 3.3).

«include» vs «extend» — a dúvida clássica

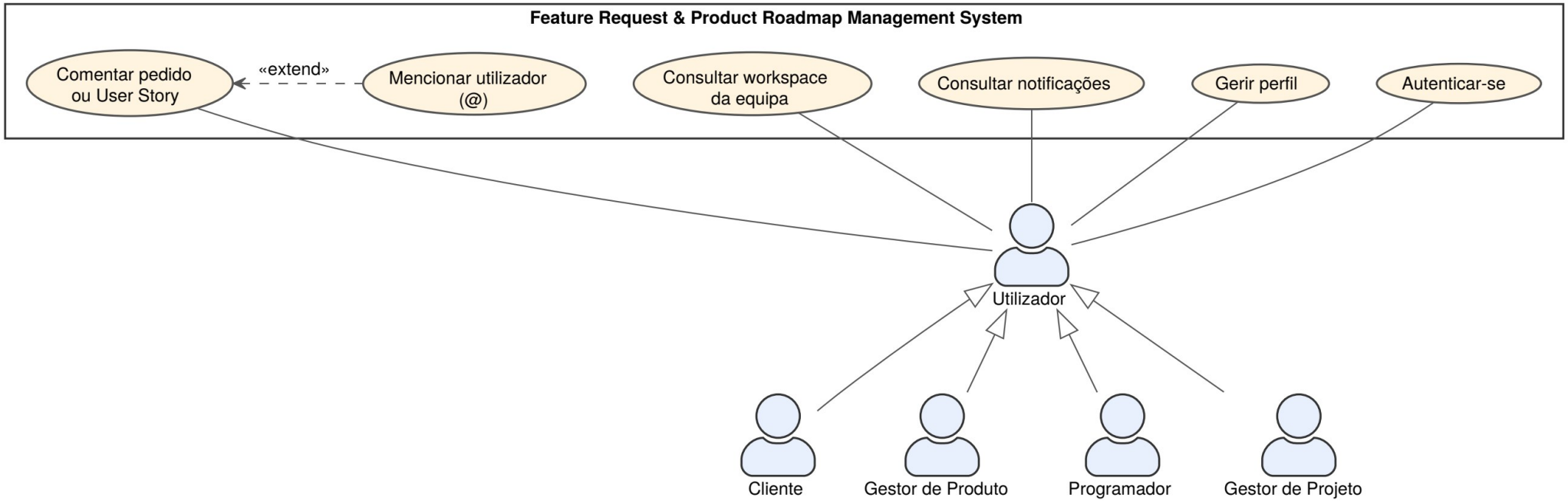
	«include»	«extend»
Quando acontece	**Sempre** que o caso base acontece	**Só às vezes** (é opcional ou condicional)
Direção da seta	Do caso base para o incluído	Do caso que estende para o caso base
O caso base faz sentido sozinho?	Não — precisa do incluído	Sim — a extensão só acrescenta
Exemplos no projeto	Aprovar pedido → Avaliar esforço e impacto Submeter pedido → Enviar confirmação por e-mail Triar pedidos → Sugerir categorias Mover User Story → Validar Definition of Ready Atribuir trabalho → Verificar capacidade Atualizar estado da US → Verificar dependências	Anexar ficheiro → Submeter pedido Retirar voto → Votar em pedido Pesquisar e filtrar → Explorar ideias Detetar duplicados → Triar · Fundir → Detetar duplicados Gerar User Story → Aprovar pedido Mencionar (@) → Comentar · Visão Executiva / Partilhar → Visualizar Roadmap

Truque para lembrar
include = "inclui sempre": aprovar *não é possível* sem avaliar (é a mesma regra da guarda [esforço e impacto definidos] no diagrama de estados e do 400 no Diagrama de Sequência 2).
extend = "extra opcional": submeter funciona sem anexo; anexar só acrescenta. A seta aponta para o caso que é estendido.

Diagrama 0 — Casos de uso comuns e hierarquia de atores

- Casos comuns: Autenticar-se, Gerir perfil, Comentar pedido ou User Story, Consultar notificações, Consultar workspace da equipa.
- Mencionar utilizador (@) **estende** Comentar (só quando se escreve @nome).
- As setas com triângulo dos 4 perfis para Utilizador são a **generalização**.

Casos de Uso Comuns e Hierarquia de Atores

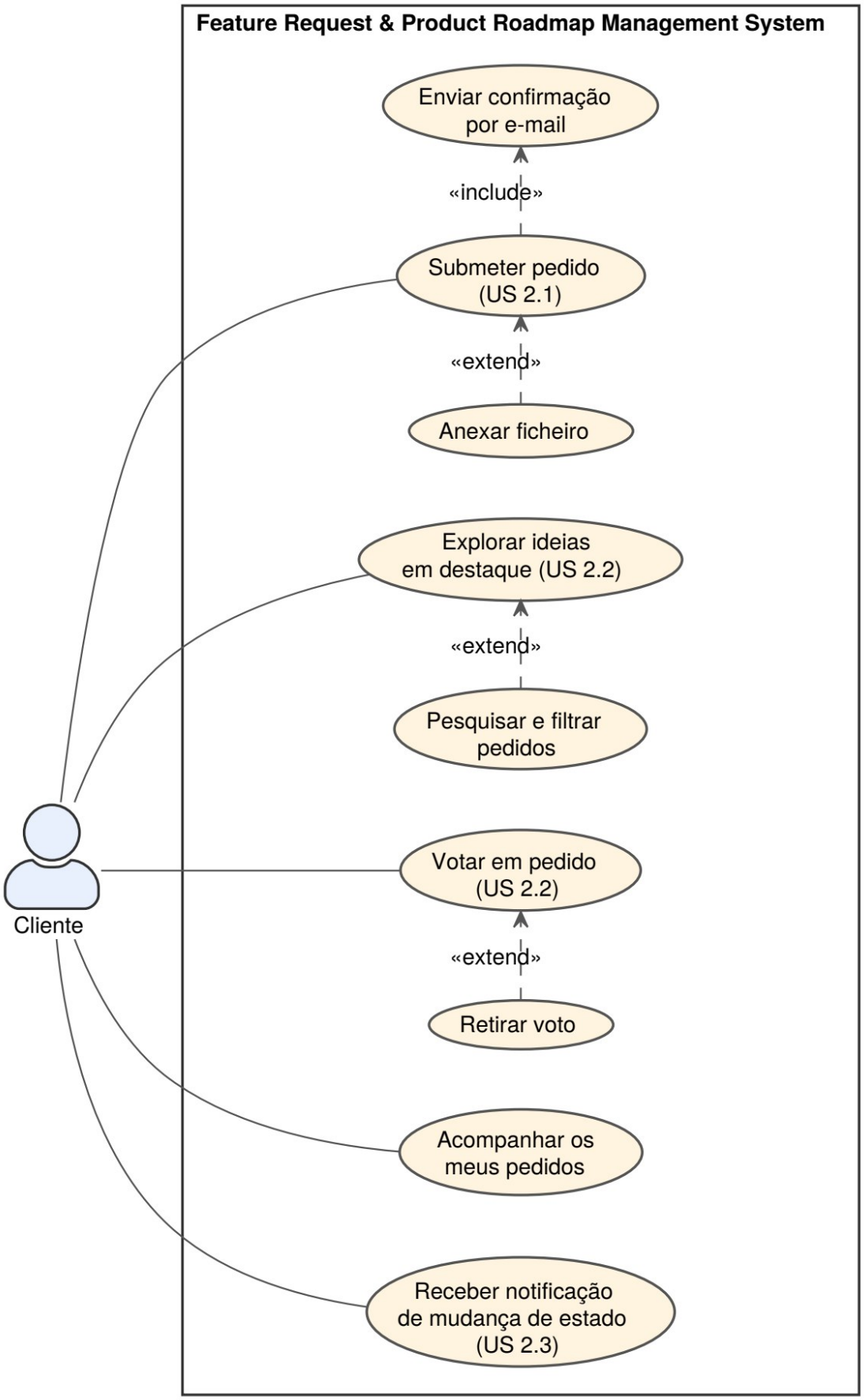


Casos de Uso comuns e hierarquia de atores (EDS-12) — no PDF podes fazer zoom: a imagem tem resolução alta.

Diagrama 1 — Cliente (Sofia)

- Associações diretas: Submeter pedido (US 2.1), Explorar ideias em destaque (US 2.2), Votar em pedido (US 2.2), Acompanhar os meus pedidos, Receber notificação de mudança de estado (US 2.3).
- Submeter **inclui** Enviar confirmação por e-mail; Anexar ficheiro **estende** Submeter.
- Pesquisar e filtrar **estende** Explorar ideias; Retirar voto **estende** Votar.

Casos de Uso — Cliente (Sofia)

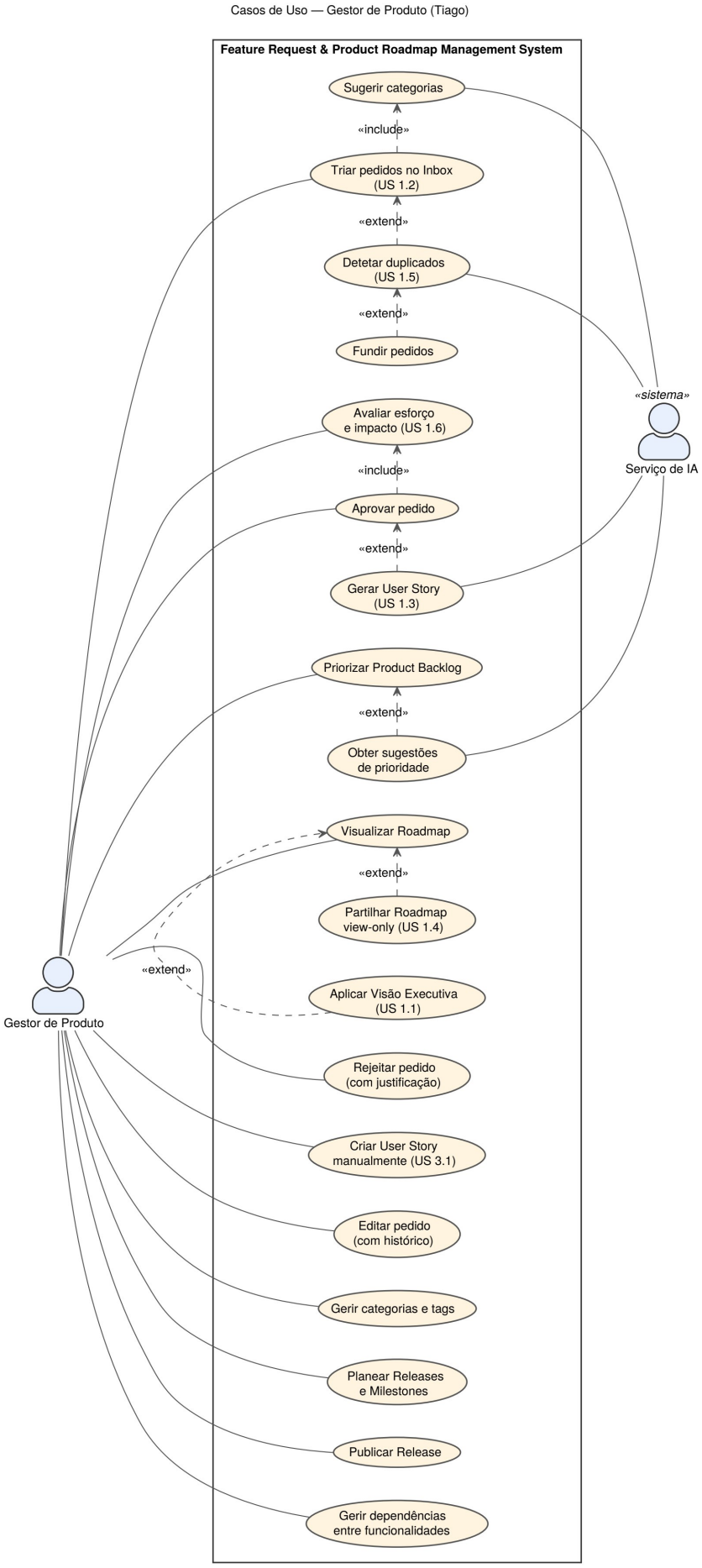


Casos de Uso — Cliente (EDS-12) — no PDF podes fazer zoom: a imagem tem resolução alta.

Diagrama 2 — Gestor de Produto (Tiago)

É o maior (19 casos de uso), porque o Gestor de Produto é quem decide sobre os pedidos e o roadmap. Agrupa-se assim:

- **Triagem:** Triar pedidos no Inbox (US 1.2) **inclui** Sugerir categorias; Detetar duplicados (US 1.5) **estende** Triar; Fundir pedidos **estende** Detetar duplicados.
- **Decisão:** Aprovar pedido **inclui** Avaliar esforço e impacto (US 1.6); Rejeitar pedido (com justificação); Gerar User Story (US 1.3) **estende** Aprovar; Criar User Story manualmente (US 3.1).
- **Gestão:** Editar pedido (com histórico), Gerir categorias e tags, Priorizar Product Backlog (Obter sugestões de prioridade **estende**).
- **Roadmap:** Planear Releases e Milestones, Publicar Release, Gerir dependências, Visualizar Roadmap (Aplicar Visão Executiva US 1.1 e Partilhar view-only US 1.4 **estendem**).
- **Serviço de IA** (ator secundário) participa em: Sugerir categorias, Detetar duplicados, Gerar User Story e Obter sugestões de prioridade — exatamente onde o C4 e os Diagramas de Sequência 1 e 2 chamam a IA.

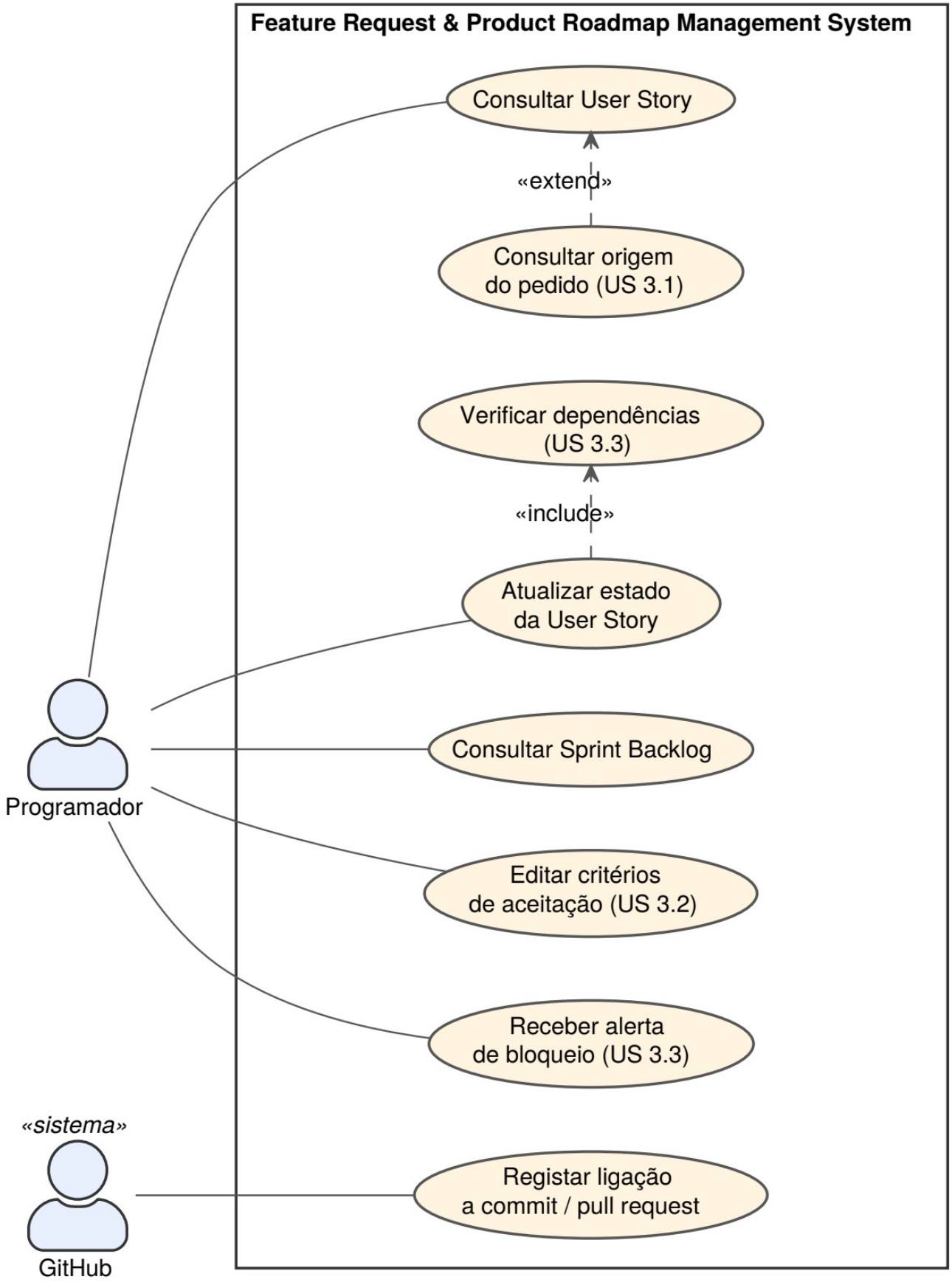


Casos de Uso — Gestor de Produto (EDS-12) — no PDF podes fazer zoom: a imagem tem resolução alta.

Diagrama 3 — Programador (Rui)

- Consultar Sprint Backlog, Consultar User Story (Consultar origem do pedido US 3.1 **estende**), Atualizar estado da User Story (**inclui** Verificar dependências, US 3.3), Editar critérios de aceitação (US 3.2), Receber alerta de bloqueio (US 3.3).
- **GitHub** (ator secundário) está ligado a Registar ligação a commit / pull request — é o webhook do C4 e o módulo integracao_github; é também o que faz a User Story passar a Em Revisão / Concluído no diagrama de estados.

Casos de Uso — Programador (Rui)

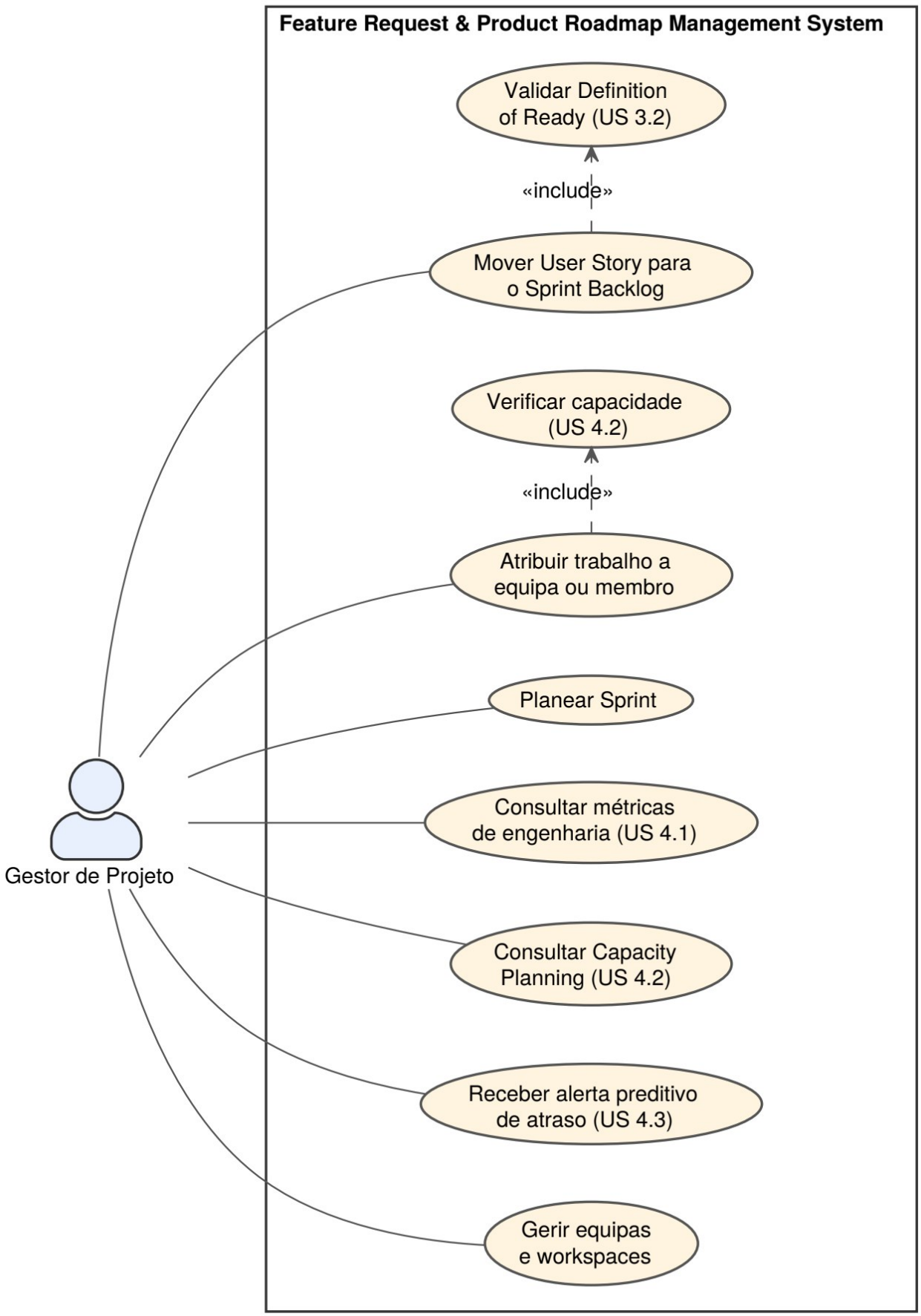


Casos de Uso — Programador (EDS-12) — no PDF podes fazer zoom: a imagem tem resolução alta.

Diagrama 4 — Gestor de Projeto (Inês)

- Planear Sprint; Mover User Story para o Sprint Backlog **inclui** Validar Definition of Ready (US 3.2) — é o Diagrama de Sequência 4.
- Atribuir trabalho a equipa ou membro **inclui** Verificar capacidade (US 4.2).
- Consultar métricas de engenharia (US 4.1), Consultar Capacity Planning (US 4.2), Receber alerta preditivo de atraso (US 4.3), Gerir equipas e workspaces.

Casos de Uso — Gestor de Projeto (Inês)



Casos de Uso — Gestor de Projeto (EDS-12) — no PDF podes fazer zoom: a imagem tem resolução alta.

Dos casos de uso aos outros diagramas

Caso de uso (EDS-12)	Realizado por	No código (Mini Sprint 3)
Autenticar-se	ADR-005 (JWT)	/auth/registo, /auth/token, /auth/eu
Submeter pedido + Anexar ficheiro	Seq 1 (passos 1–9), Atividade	POST /pedidos (multipart)
Enviar confirmação por e-mail («include»)	Seq 1, Worker	Enviado logo após a submissão (pedidos/services.py); passa para o Worker no Sprint 4
Pesquisar e filtrar / Acompanhar os meus pedidos	Wireframe "Pedidos"	Filtros e separadores "Os meus pedidos" e "Votados por mim"
Votar em pedido / Retirar voto	Seq 3	POST e DELETE /pedidos/{id}/votos
Triar, Detetar duplicados, Fundir	Seq 1 (passos 10–34)	Sprint 5
Avaliar esforço e impacto + Aprovar pedido	Seq 2 (passos 1–10)	PATCH /pedidos/{id}, POST /aprovar
Rejeitar pedido (com justificação)	Seq 1 (passos 35–38)	POST /rejeitar
Gerar User Story	Seq 2 (passos 11–31)	Sprint 4
Mover User Story + Validar DoR	Seq 4	Sprint 4

Perguntas que te podem fazer

P: Qual a diferença entre «include» e «extend»?

R: include acontece sempre (aprovar inclui avaliar); extend é opcional (anexar ficheiro estende submeter). A seta do include vai do caso base para o incluído; a do extend vai do caso opcional para o base.

P: Porque é que o Serviço de IA e o GitHub são atores?

R: Porque estão fora do sistema e interagem com ele. São atores secundários (marcados «sistema»): o sistema recorre a eles para cumprir casos de uso do Gestor de Produto e do Programador. São os mesmos sistemas externos do C4 nível 1.

P: Porque é que há um ator Utilizador?

R: Para não repetir os casos comuns em cada persona: os 4 perfis especializam Utilizador (generalização) e herdam autenticar-se, comentar, notificações, etc.

P: "Enviar confirmação por e-mail" é um caso de uso?

R: É um comportamento do sistema que acontece sempre ao submeter, por isso foi modelado como «include» de Submeter pedido. Uma crítica possível é que não é um objetivo do ator; se perguntarem, explica que foi separado para mostrar que é obrigatório e que é feito pelo Worker (ADR-004).

4. Arquitetura — Modelo C4 (EDS-10 e EDS-11)

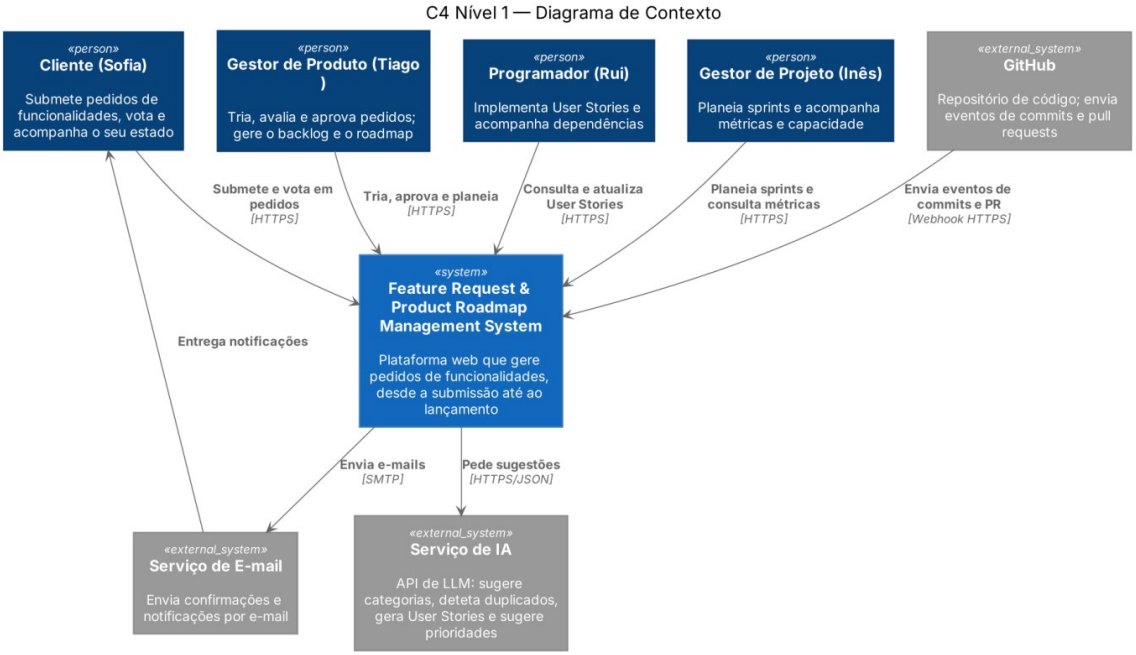
O C4 descreve a arquitetura por níveis de zoom, como um mapa: primeiro o país, depois as cidades, depois as ruas. O estilo escolhido (ADR-001) é **cliente-servidor em camadas com monólito modular**.

Nível 1 — Contexto

- **Pessoas (4 personas):** Cliente (Sofia), Gestor de Produto (Tiago), Programador (Rui), Gestor de Projeto (Inês).
- **O sistema:** uma única caixa — o FeatureFlow.
- **Sistemas externos:** Serviço de IA (LLM), GitHub (eventos de commits e PR por webhook) e Serviço de E-mail (SMTP).
- Cada seta diz o que a pessoa faz no sistema ("Submete e vota em pedidos") e o protocolo (HTTPS).

Nível 1 — Contexto

Mostra o sistema como uma única caixa, quem o usa (as quatro personas) e os sistemas externos com que comunica: o Serviço de IA, o GitHub e o Serviço de E-mail.



1

C4 nível 1 — Contexto (EDS-11) — no PDF podes fazer zoom: a imagem tem resolução alta.

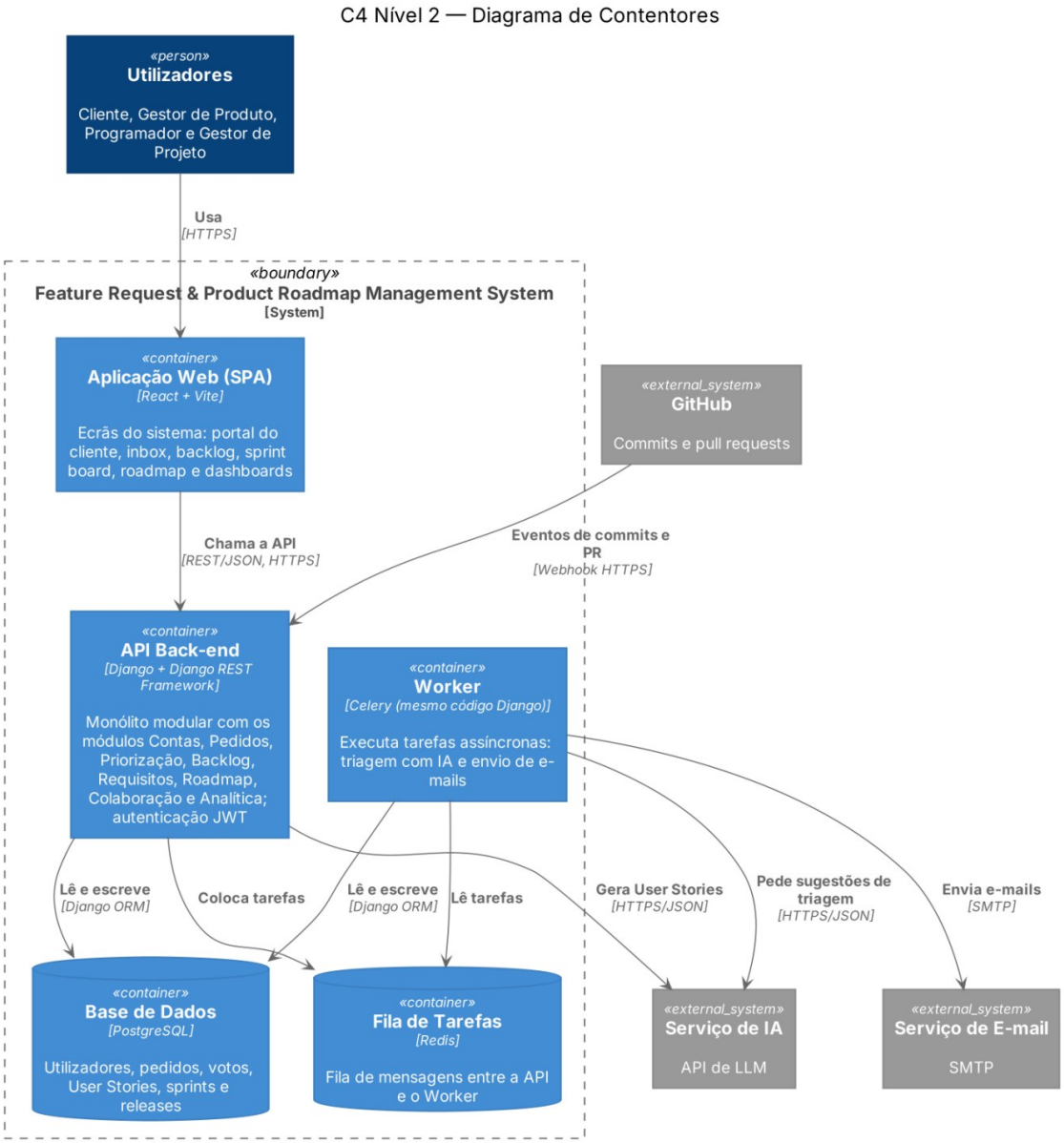
Nível 2 — Contentores

Contentor	Tecnologia	Função
Aplicação Web (SPA)	React + Vite	Os ecrãs; só fala com a API por REST/JSON
API Back-end	Django + DRF	Monólito modular; autenticação JWT; regras de negócio
Worker	Celery (mesmo código Django)	Trabalho lento: triagem com IA e envio de e-mails
Fila de Tarefas	Redis	Passa tarefas da API para o Worker
Base de Dados	PostgreSQL	Utilizadores, pedidos, votos, stories, sprints, releases

Repara em **quem chama a IA**: o Worker pede a triagem (assíncrona, o cliente não espera) e a API pede o esboço de User Story (síncrono, porque o Gestor espera para rever). Esta diferença vem do ADR-004.

Nível 2 — Contentores

Abre o sistema e mostra as peças que correm separadamente: a aplicação web em React, a API em Django REST Framework (monólito modular), o Worker (Celery) com a fila de tarefas Redis para o trabalho assíncrono, e a base de dados PostgreSQL, com a tecnologia e o protocolo de cada ligação.

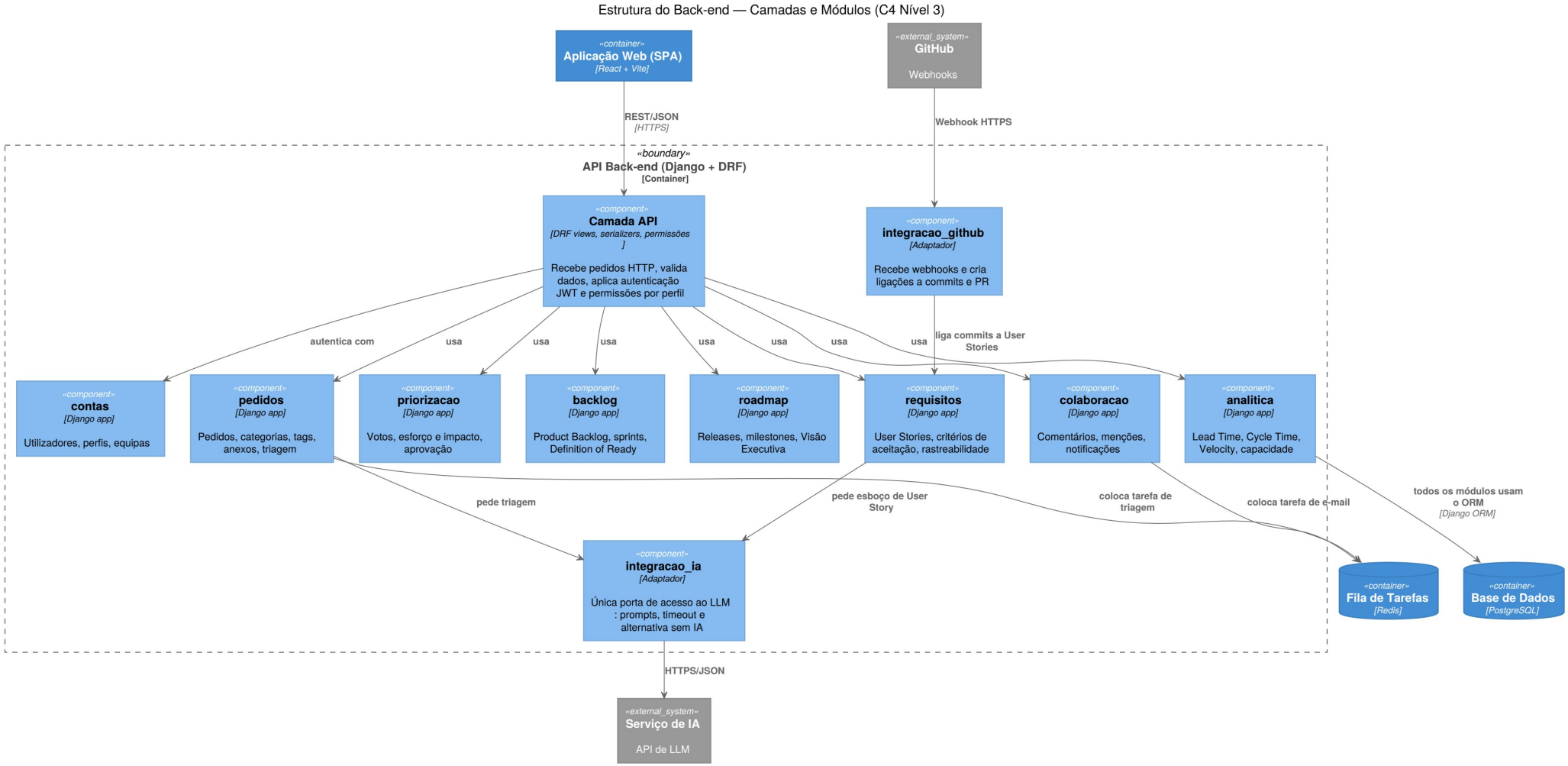


C4 nível 2 — Contentores (EDS-11) — no PDF podes fazer zoom: a imagem tem resolução alta.

Nível 3 — Componentes do back-end

Dentro da API há uma **Camada API** (views, serializers, permissões) que usa os **módulos** (apps Django), um por área funcional: contas, pedidos, priorizacao, backlog, requisitos, roadmap, colaboracao, analitica, e os adaptadores integracao_ia e integracao_github.

- **Quatro camadas** (EDS-10 §3.1): API → serviços de domínio → acesso a dados (ORM) → adaptadores de integração. Cada camada só conhece a de baixo.
- **Regra de dependências:** um módulo só usa outro através das suas funções de serviço, nunca os modelos internos.
- **Adaptador de IA** (ADR-003): única porta para o LLM, com timeout e alternativa sem IA (RNF-07).



C4 nível 3 — Componentes do back-end (EDS-10 e EDS-11) — no PDF podes fazer zoom: a imagem tem resolução alta.

Os ADRs que explicam o C4

ADR	Decisão	Porquê (resumo)
001	Cliente-servidor em camadas, monólito modular	5 pessoas, 12 semanas; microserviços seriam complexidade sem benefício
002	React + Vite, Django + DRF, PostgreSQL, Docker Compose, GitHub Actions	Django traz ORM, migrações, auth e admin; PostgreSQL suporta CHECK e chaves compostas
003	IA através de um adaptador (integracao_ia)	Trocar de fornecedor só muda um módulo; falhas tratadas igual em todo o lado
004	Celery + Redis para trabalho assíncrono	Submeter nunca falha por causa da IA ou do e-mail
005	JWT + permissões por perfil	SPA e API separadas; regras de acesso num só sítio
006	REST em JSON com prefixo /api/v1/	Contrato estável entre front-end e back-end
007	Dashboards por polling a cada 15 s	Cumpre RNF-02 (< 30 s) sem WebSockets
008	Claude Code como ferramenta de IA para programar	Restrição 7 do enunciado (proposto no Mini Sprint 3)

No código do Mini Sprint 3

Contentores: `docker-compose.yml` arranca db (PostgreSQL), backend (Django) e frontend (Vite). Worker e Redis ainda não existem — entram com a triagem por IA (Sprint 5).
Módulos: `backend/contas`, `backend/pedidos`, `backend/priorizacao`. Camada de serviços: `priorizacao/services.py` (votar, retirar_voto, avaliar).

Perguntas que te podem fazer

P: Porque não microserviços?

> **R:** Exigiriam várias bases de dados, comunicação pela rede e orquestração, sem benefício para o volume do projeto (ADR-001). O monólito modular mantém módulos separados e permite extrair um no futuro.

P: Qual a diferença entre contentor e componente?

> **R:** Um contentor corre separadamente (processo/aplicação: SPA, API, Worker, BD). Um componente é uma parte dentro de um contentor (um módulo Django dentro da API).

P: O que acontece se a IA estiver em baixo?

> **R:** O adaptador devolve a alternativa sem IA: o pedido fica "Por classificar" e o Gestor classifica à mão (RNF-07, ADR-003).

5. Diagrama de Classes (EDS-13)

Mostra as **14 classes e 3 enumerações** do domínio, com atributos, as operações que são regras de negócio e as relações. É a base dos modelos Django e do modelo ER.

Como está montado

As classes estão agrupadas pelos módulos do back-end (EDS-10):

Área (módulo)	Classes
Utilizadores e equipas (contas)	Utilizador, Equipa
Pedidos e priorização (pedidos, priorizacao)	Pedido, Categoria, Tag, Anexo, Voto
Colaboração (colaboracao)	Comentario, Notificacao
Backlog e requisitos (backlog, requisitos)	UserStory, CriterioAceitacao, Sprint
Roadmap (roadmap)	Release, Milestone
Enumerações	Perfil, EstadoPedido, EstadoUserStory

Como ler o percurso principal: um Utilizador *submete* um Pedido → clientes *votam* (Voto) e comentam → o Pedido aprovado *origina* User Stories → cada story *tem* Critérios de Aceitação → as stories entram num Sprint → o Pedido é *incluído* numa Release.

As relações mais importantes

Relação	Multiplicidade	O que significa
Utilizador — submete — Pedido	1 — *	Cada pedido tem exatamente um autor
Utilizador — dá — Voto — recebe — Pedido	1 — * — 1	Voto é classe própria que liga quem votou ao pedido
Pedido — pertence a — Categoria	* — 0..1	Categoria opcional (o cliente não a escolhe; a IA sugere)
Pedido □— Anexo	1 — *	Composição: apagar o pedido apaga os anexos
Pedido — tem etiquetas — Tag	* — *	Muitos-para-muitos → tabela pedido_tag no ER
Pedido — origina — UserStory	0..1 — *	Story pode ser criada à mão, sem pedido
UserStory □— CriterioAceitacao	1 — 0..*	Composição; pode não ter critérios no backlog
UserStory — depende de — UserStory	* — *	Auto-associação (dependências) → tabela dependencia
Sprint — contém — UserStory	0..1 — *	Sem sprint = está no Product Backlog
Release □— Milestone	1 — *	Composição
Comentario — Pedido / UserStory	0..1 cada	Pertence a exatamente um dos dois (nota + CHECK no ER)

As 15 decisões de design (decora as principais)

#	Decisão	Porquê
3.1	Só operações de negócio no diagrama	Fixa o vocabulário; getters/setters mudariam a cada sprint

#	Decisão	Porquê
3.2	Pedido ≠ UserStory	Ciclos de vida e utilizadores diferentes; um pedido pode dar várias stories
3.3	Perfil é atributo, não subclasse	Os 4 perfis têm os mesmos dados, só mudam permissões
3.4	Estados como enumerações	Conjunto fechado; cada valor = um estado do diagrama de estados
3.5	Voto é classe própria	Saber quem votou: impedir repetidos, retirar, notificar
3.6	Composição para Anexo e Critério	Não fazem sentido sem o dono
3.7	Um só Comentario	Mesma estrutura para pedidos e stories
3.8	Auto-associação de dependências	Calcular bloqueios automaticamente (estaBloqueada())
3.9	Datas e capacidadeSP	Lead Time, Cycle Time e alerta de sobrecarga
3.10	Product Backlog sem classe	É o conjunto de stories sem sprint, ordenadas por ordemBacklog
3.11	Roadmap ao nível do Pedido	Direção vê funcionalidades, não tarefas técnicas
3.12	Categoria opcional	O formulário do cliente só tem 2 campos
3.13	justificacaoRejeicao; Equipa * — * Utilizador	Guardar a razão; uma pessoa em várias equipas
3.14	Tag * — *; fundirCom()	Várias etiquetas por pedido; fusão em vez de estado Duplicado
3.15	IA, histórico e links fora do diagrama	IA é sistema externo; sugestões num campo JSON

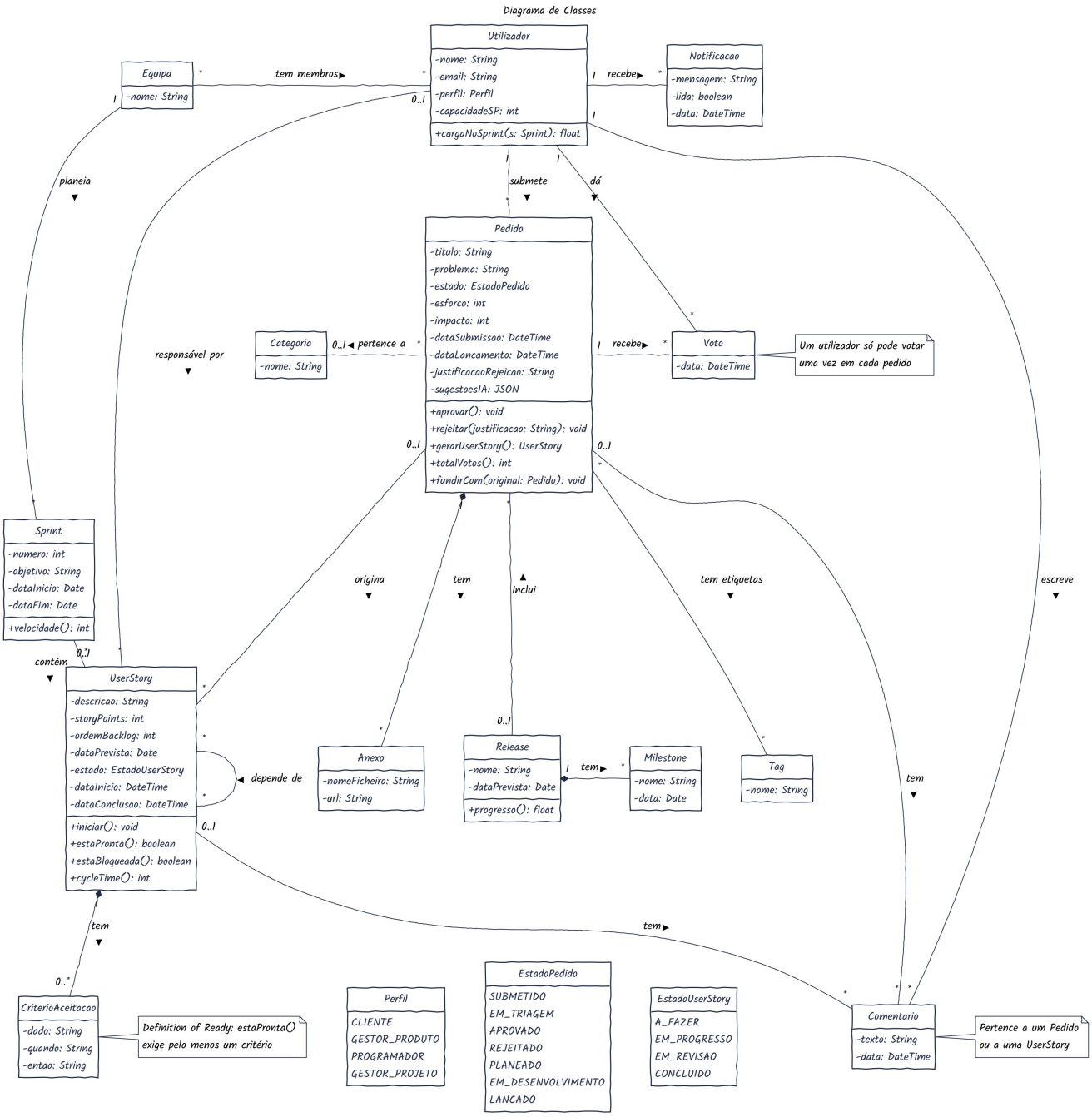


Diagrama de Classes (EDS-13) — no PDF podes fazer zoom: a imagem tem resolução alta.

No código do Mini Sprint 3

`Utilizador` e `Perfil` → `backend/contas/models.py` (perfil com `choices`).
`Pedido`, `Categoria`, `Anexo`, `EstadoPedido` → `backend/pedidos/models.py`, com `aprovar()`, `rejeitar(justificacao)` e `total_votos()`.
`Voto` → `backend/priorizacao/models.py`. As outras classes entram nos Sprints 4 e 5.

Perguntas que te podem fazer

P: Porque é que o Voto não é só um contador no Pedido?

R: Porque é preciso saber quem votou: para impedir votos repetidos, permitir retirar o voto e notificar quem votou (decisão 3.5, US 2.2 e 2.3). O total calcula-se com totalVotos().

P: Qual a diferença entre composição e associação?

R: Na composição (losango preto) a parte não existe sem o todo e é apagada com ele (Anexo de um Pedido). Na associação os objetos vivem independentemente (Pedido e Categoria).

P: Porque é que UserStory — CriterioAceitacao é 0..* e não 1..*?

R: Uma story pode ficar no backlog sem critérios (ex.: se a IA falhar). A exigência de pelo menos um só se aplica ao entrar num Sprint, via estaPronta() (Definition of Ready).

P: Porque é que o perfil não é herança?

R: Os quatro perfis têm exatamente os mesmos dados; só diferem em permissões, tratadas na autenticação. Um atributo permite mudar o perfil sem recriar o objeto.

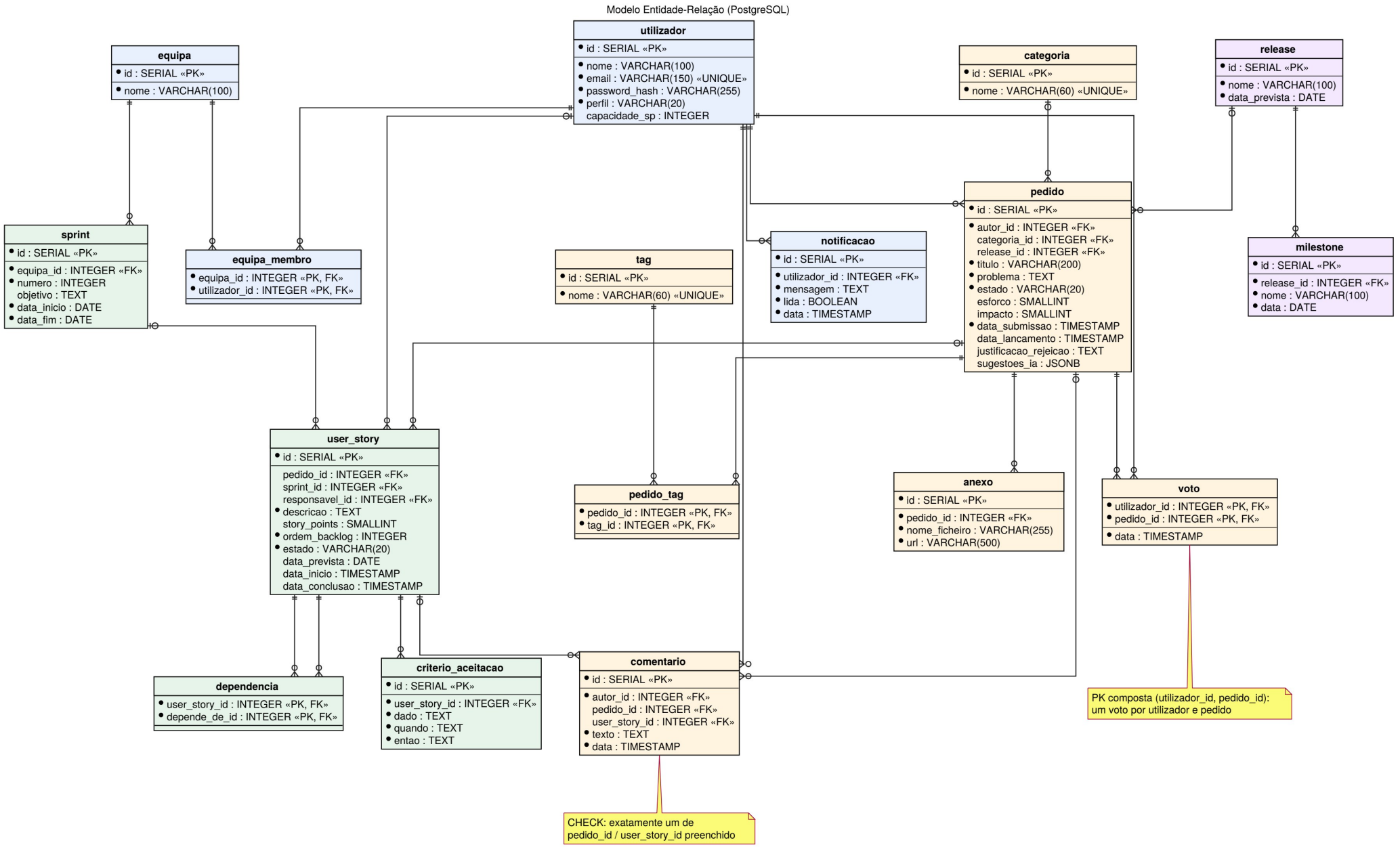
6. Modelo Entidade-Relação (EDS-14)

É o diagrama de classes traduzido para tabelas PostgreSQL. A tradução segue três regras simples:

- **Cada classe → uma tabela** (Pedido → `pedido`, UserStory → `user_story`). Os atributos em camelCase passam a snake_case (`dataSubmissao` → `data_submissao`).
- **Cada associação um-para-muitos → chave estrangeira** do lado "muitos" (Pedido tem `autor_id`, `categoria_id`, `release_id`).
- **Cada associação muitos-para-muitos → tabela de associação** com chave primária composta: `equipa_membro`, `pedido_tag` e `dependencia`.
- **Enumerações → VARCHAR com valores fixos** (perfil, estado).

Regras garantidas pela base de dados

Regra	Como	Requisito
Um voto por utilizador e pedido	PK composta (utilizador_id, pedido_id) na tabela voto	US 2.2
Comentário pertence a um só dono	CHECK: exatamente um de pedido_id / user_story_id	Decisão 3.7
Emails, categorias e tags únicos	UNIQUE em email, categoria.nome, tag.nome	—
Story pode não ter pedido	user_story.pedido_id opcional	US 3.1
Anexos e critérios apagados com o dono	ON DELETE CASCADE	Decisão 3.6
Ordem do backlog sempre preenchida	ordem_backlog NOT NULL (último lugar ao criar)	Decisão 3.10
Sugestões da IA	Coluna sugestoes_ia do tipo JSONB	Decisão 3.15



Modelo Entidade-Relação (EDS-14) — no PDF podes fazer zoom: a imagem tem resolução alta.

No código do Mini Sprint 3

As tabelas têm os mesmos nomes do ER (`db_table = "pedido"`, "voto", "utilizador"…) e os IDs são SERIAL (AutoField). A coluna da palavra-passe chama-se `password_hash` como no ER.
O voto tem a **PK composta (utilizador_id, pedido_id)** como no ER (`models.CompositePrimaryKey` do Django 5.2): a BD garante um voto por utilizador e pedido.

Perguntas que te podem fazer

P: Como se passa de uma associação muitos-para-muitos para tabelas?

R: Cria-se uma tabela de associação com as duas FKs, que juntas formam a chave primária. Ex.: Pedido * — * Tag dá pedido_tag(pedido_id, tag_id).

P: Porque é que a regra do voto está na BD e não só na interface?

R: Porque a interface pode ser contornada (pedido direto à API, dois cliques ao mesmo tempo). A restrição na BD garante a regra sempre.

P: Como se lê o pé-de-galinha entre pedido e anexo?

R: || do lado do pedido (cada anexo tem exatamente um pedido) e o{ do lado do anexo (um pedido tem zero ou mais anexos).

7. Diagramas de Estados

Mostram o **ciclo de vida** dos dois objetos centrais. Os estados são exatamente os valores das enumerações `EstadoPedido` e `EstadoUserStory`, e as transições são as operações da classe.

Diagrama 1 — Pedido

De	Para	Evento [guarda] / ação
(início)	Submetido	submeter() — entry / guardar dataSubmissao
Submetido	Em Triagem	triagem concluída pelo Worker (com ou sem sugestões da IA)
Em Triagem	Aprovado	aprovar() **[esforço e impacto definidos]**
Em Triagem	Rejeitado → fim	rejeitar(justificacao)
Em Triagem	fim	fundirCom(original) / transferir votos e anexos
Aprovado	Planeado	associado a uma Release / notificar cliente
Planeado	Em Desenvolvimento	1.ª User Story passa a EM_PROGRESSO / notificar cliente
Em Desenvolvimento	Lançado → fim	Release publicada / notificar cliente — entry / guardar dataLancamento

- **Três saídas da triagem:** aprovar, rejeitar ou fundir (o duplicado deixa de existir: não há estado "Duplicado").
- **Notificações:** só nas passagens para Planeado, Em Desenvolvimento e Lançado (US 2.3).
- **Datas guardadas nas transições:** dataSubmissao e dataLancamento dão o Lead Time.

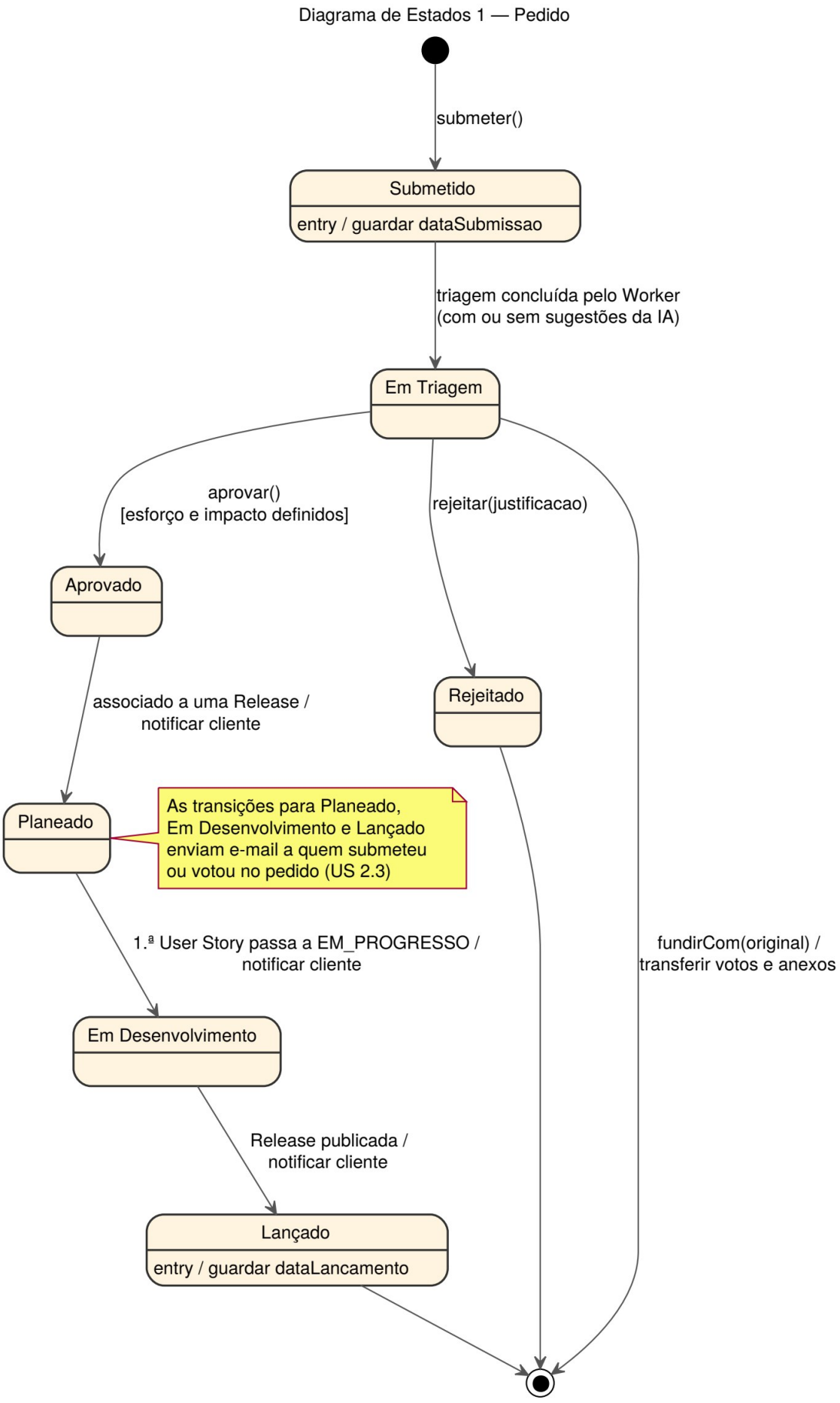


Diagrama de Estados 1 — Pedido — no PDF podes fazer zoom: a imagem tem resolução alta.

Diagrama 2 — User Story

De	Para	Evento [guarda] / ação
(início)	A Fazer	gerarUserStory()
A Fazer	Em Progresso	iniciar() **[estaPronta() e está num Sprint]** — entry / guardar dataInicio; do / verificar dependências
Em Progresso	Em Revisão	pull request aberto
Em Revisão	Em Progresso	alterações pedidas
Em Revisão	Concluído → fim	pull request aprovado — entry / guardar dataConclusao

- **estaPronta()** = tem pelo menos um critério de aceitação (Definition of Ready, US 3.2).
- **Bloqueio não é estado:** uma story bloqueada continua Em Progresso e é só sinalizada com estaBloqueada().
- **Ligação entre os dois ciclos:** a 1.ª story em Em Progresso faz o Pedido passar a Em Desenvolvimento.
- dataInicio e dataConclusao dão o Cycle Time.

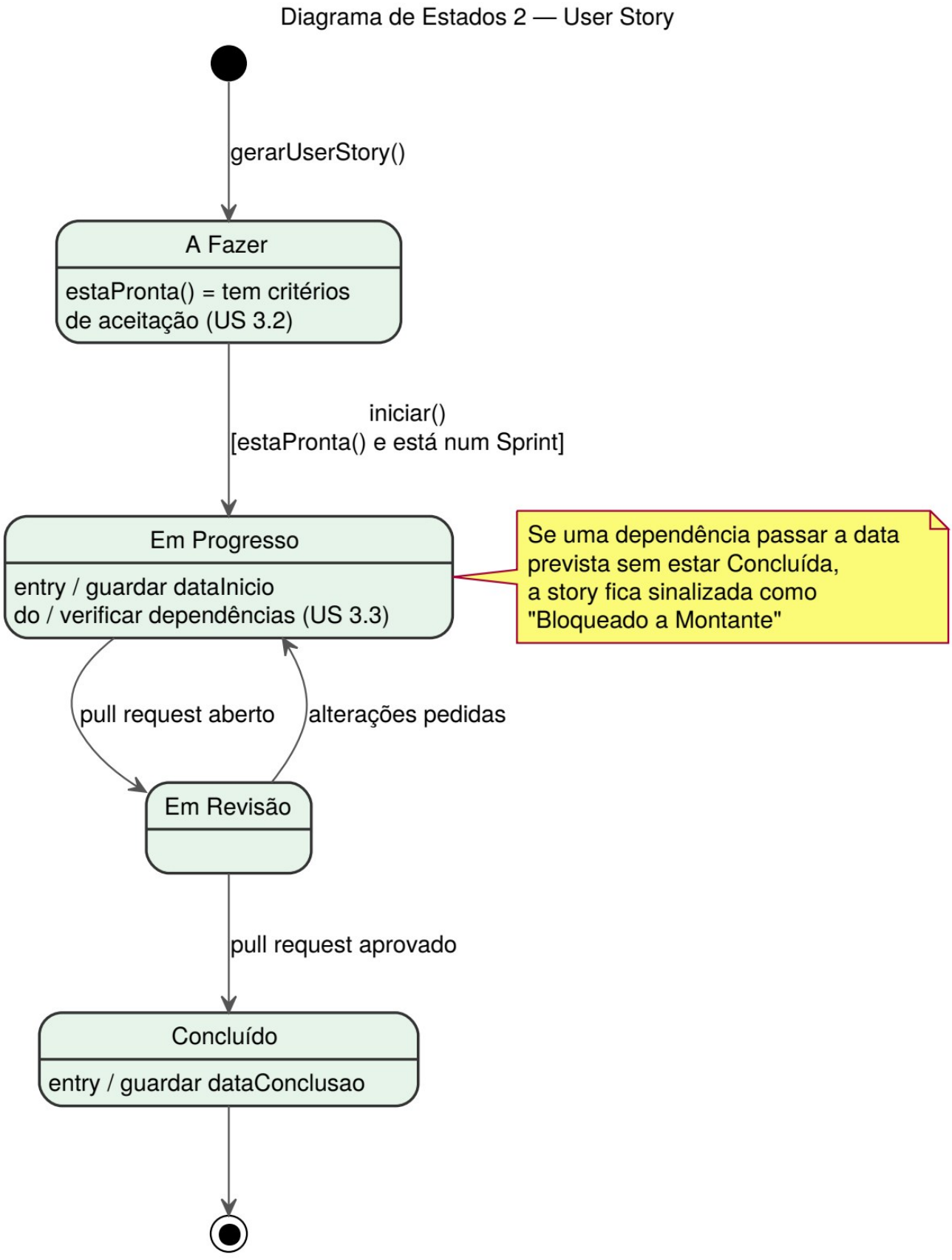


Diagrama de Estados 2 — User Story — no PDF podes fazer zoom: a imagem tem resolução alta.

No código do Mini Sprint 3

`EstadoPedido` tem os 7 valores do diagrama (`pedidos/models.py`). `aprovar()` e `rejeitar()` lançam `TransicaoInvalida` quando a guarda falha (a API responde 400).
Submetido → Em Triagem: `concluir_triagem()`, chamado logo após a submissão (ainda sem IA, o caso "timeout ou erro da IA" do Seq 1). `aprovar()` e `rejeitar()` só aceitam EM_TRIAGEM, como no diagrama.

Perguntas que te podem fazer

P: O que é uma guarda?

R: Uma condição entre [ ] que tem de ser verdadeira para a transição acontecer. Ex.: aprovar() só passa a Aprovado se [esforço e impacto definidos].

P: Porque é que "Bloqueada" não é um estado da User Story?

R: Porque o trabalho pode continuar noutras partes; o bloqueio é um sinal calculado (estaBloqueada()) e não muda a fase do trabalho.

P: Qual a diferença entre entry e do?

R: entry executa uma vez ao entrar no estado (guardar a data); do é uma atividade que decorre enquanto o objeto está no estado (verificar dependências).

8. Diagramas de Sequência 1–4

Cada diagrama é **um cenário** contado como mensagens entre participantes ao longo do tempo. Os participantes são **os contentores do C4 nível 2** (Aplicação Web, API, Base de Dados, Fila, Worker, IA, E-mail) e os endpoints são os do ADR-006. Todos seguem o mesmo esqueleto de documento: Objetivo → Participantes → Descrição do fluxo por passos → Decisões e regras → Ligação aos outros artefactos → Diagrama → Código PlantUML.

#	Cenário	Atores	US	Fragmentos
1	Submeter e triar pedido com IA	Cliente, Gestor de Produto	2.1, 1.2, 1.5	alt (validação, IA, 3 decisões)
2	Aprovar pedido e gerar User Story	Gestor de Produto	1.6, 1.3	alt, loop (2 tentativas)
3	Votar e retirar voto	Cliente	2.2	alt aninhados
4	Mover User Story para o Sprint	Gestor de Projeto	3.2	alt, opt

Sequência 1 — Submeter e triar pedido com IA

Está dividido em três fases (separadores ==):

- **Submissão (1–9):** POST /pedidos. **alt**: campos em falta → 400; válidos → guarda em SUBMETIDO, coloca 2 tarefas na fila (e-mail e triagem) e responde 201 logo — o cliente não espera pela IA.
- **Processamento assíncrono (10–18):** o Worker envia o e-mail, procura semelhantes, chama sugerirCategorias(). **alt**: IA responde → guarda em sugestoesIA; timeout/erro → "Por classificar". Em ambos passa a EM_TRIAGEM.
- **Revisão pelo Gestor (19–39):** GET /pedidos/{id} e **alt** com três ramos: aceitar categorias (PATCH), fundir duplicado (POST /fundir → fundirCom()) ou rejeitar com justificação (POST /rejeitar).

Regras: a IA sugere, o Gestor decide; triagem e e-mail são assíncronos (ADR-004, RNF-07); fusão em vez de estado Duplicado.

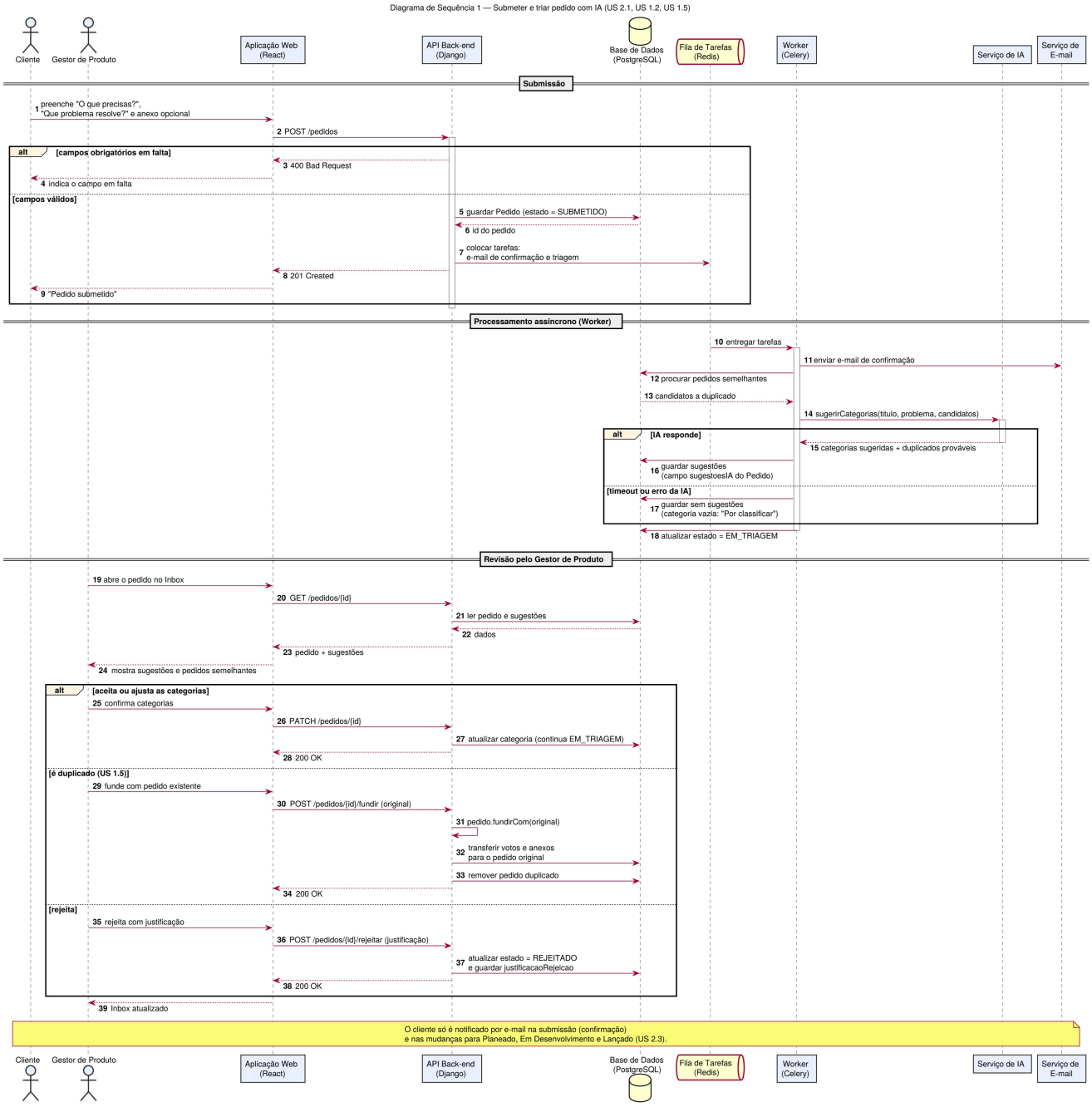


Diagrama de Sequência 1 — Submeter e triar pedido com IA — no PDF podes fazer zoom: a imagem tem resolução alta.

Sequência 2 — Aprovar pedido e gerar User Story

- **Aprovação (1–10):** POST /pedidos/{id}/aprovar. **alt**: falta esforço ou impacto → 400 "Avaliação obrigatória"; caso contrário pedido.aprovar() e estado APROVADO.
- **Geração (11–22):** POST /pedidos/{id}/user-stories → gerarUserStory(). **loop** até 2 tentativas à IA ("Como…, eu preciso de…, para que…" + Dado/Quando/Então). Se falhar, usa a descrição original sem critérios. A story fica A_FAZER, no último lugar do backlog e ligada ao pedido.
- **Revisão (23–31): alt** editar e confirmar (PUT) ou descartar (DELETE → 204).

Regras: avaliação obrigatória antes de aprovar (US 1.6); o utilizador recebe sempre uma story, nunca um erro sem saída.

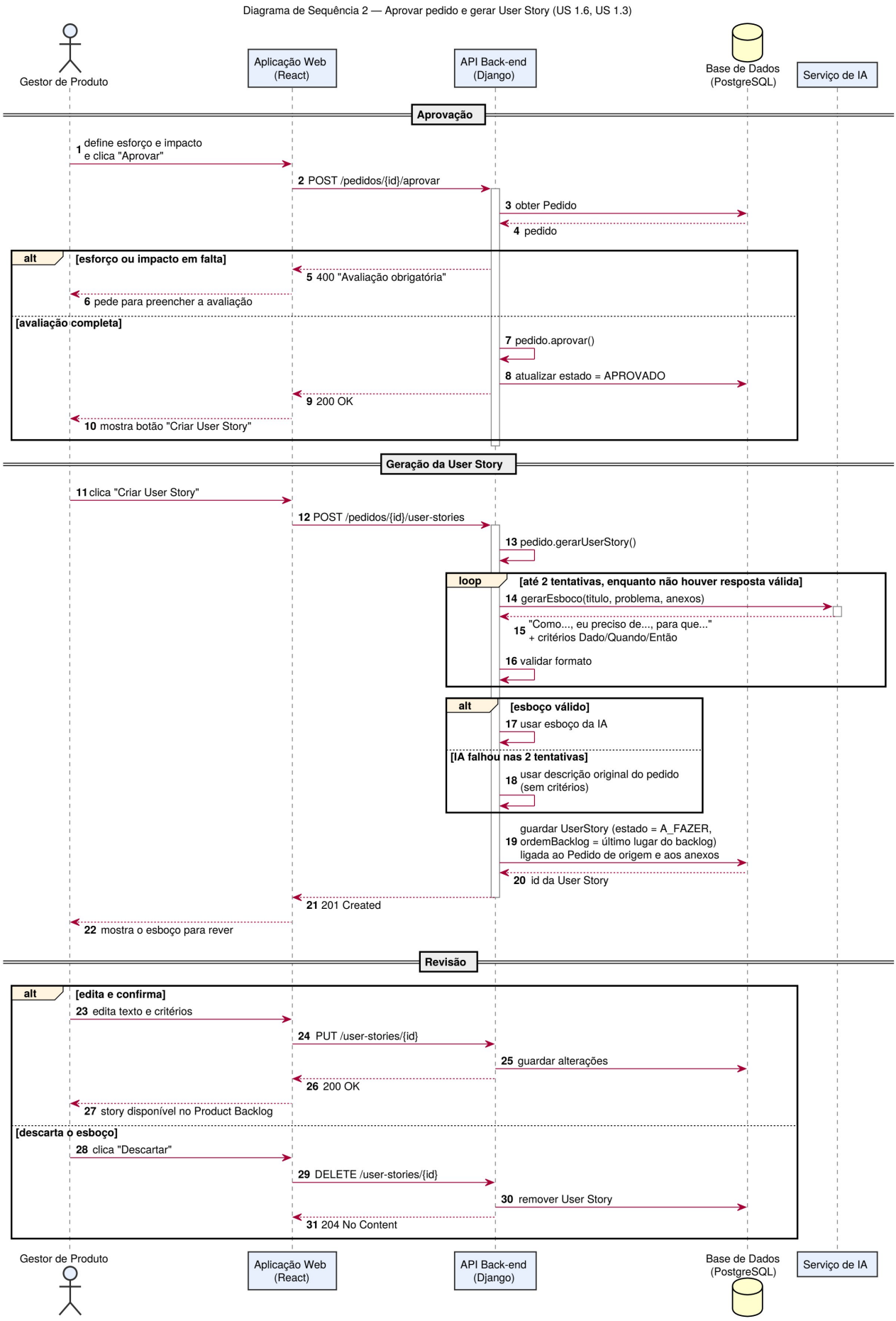


Diagrama de Sequência 2 — Aprovar pedido e gerar User Story — no PDF podes fazer zoom: a imagem tem resolução alta.

Sequência 3 — Votar e retirar voto

- **1–3:** o Cliente clica "Apoiar". **alt**: ainda não votou → POST /pedidos/{id}/votos; já votou → DELETE.
- **4–10 (validações comuns):** token inválido → 401 e vai para o login; pedido REJEITADO ou LANCADO → 403 (votação fechada).
- **11–16: alt** inserir ou remover o Voto (chave única), contar votos e devolver o novo total; o botão alterna entre "Apoiar" e "Apoiado".

Regras: um voto por utilizador e pedido garantido pela BD; total sempre contado (totalVotos()), sem contador guardado; a área de acompanhamento do cliente é calculada a partir dos votos.

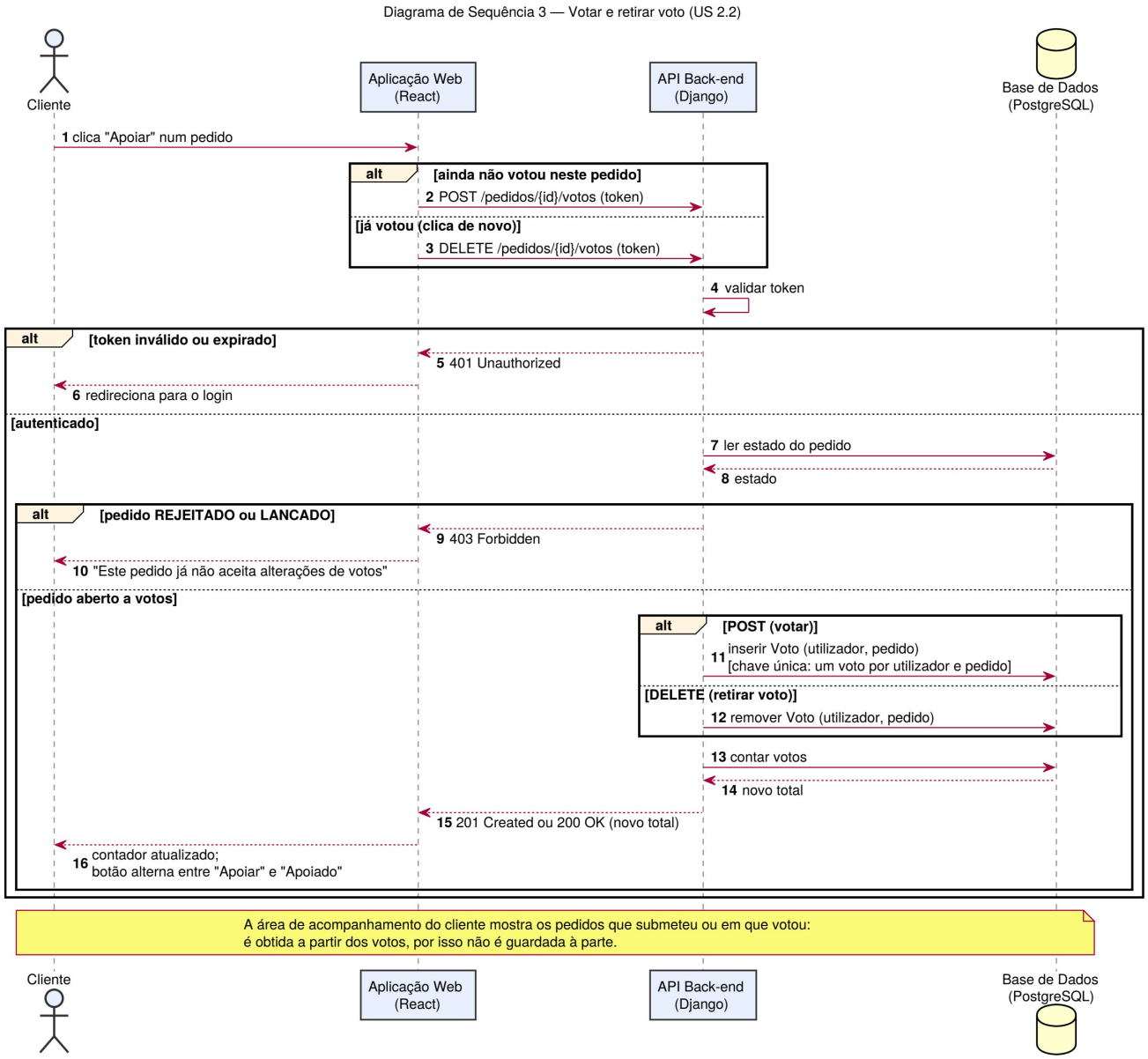


Diagrama de Sequência 3 — Votar e retirar voto — no PDF podes fazer zoom: a imagem tem resolução alta.

Sequência 4 — Mover User Story para o Sprint (Definition of Ready)

- **1–10:** o Gestor de Projeto arrasta a story; POST /sprints/{sid}/user-stories; a API executa story.estaPronta(). **alt**: verdadeiro → associa ao Sprint (estado continua A_FAZER), 200; falso → **422** "Definition of Ready não cumprida: faltam Critérios de Aceitação".
- **11–15 (opt):** acrescenta um critério Dado/Quando/Então (POST /user-stories/{id}/criterios) e tenta de novo.

Regras: a regra é validada no servidor, não só na interface; entrar no Sprint não muda o estado (só iniciar() o faz).

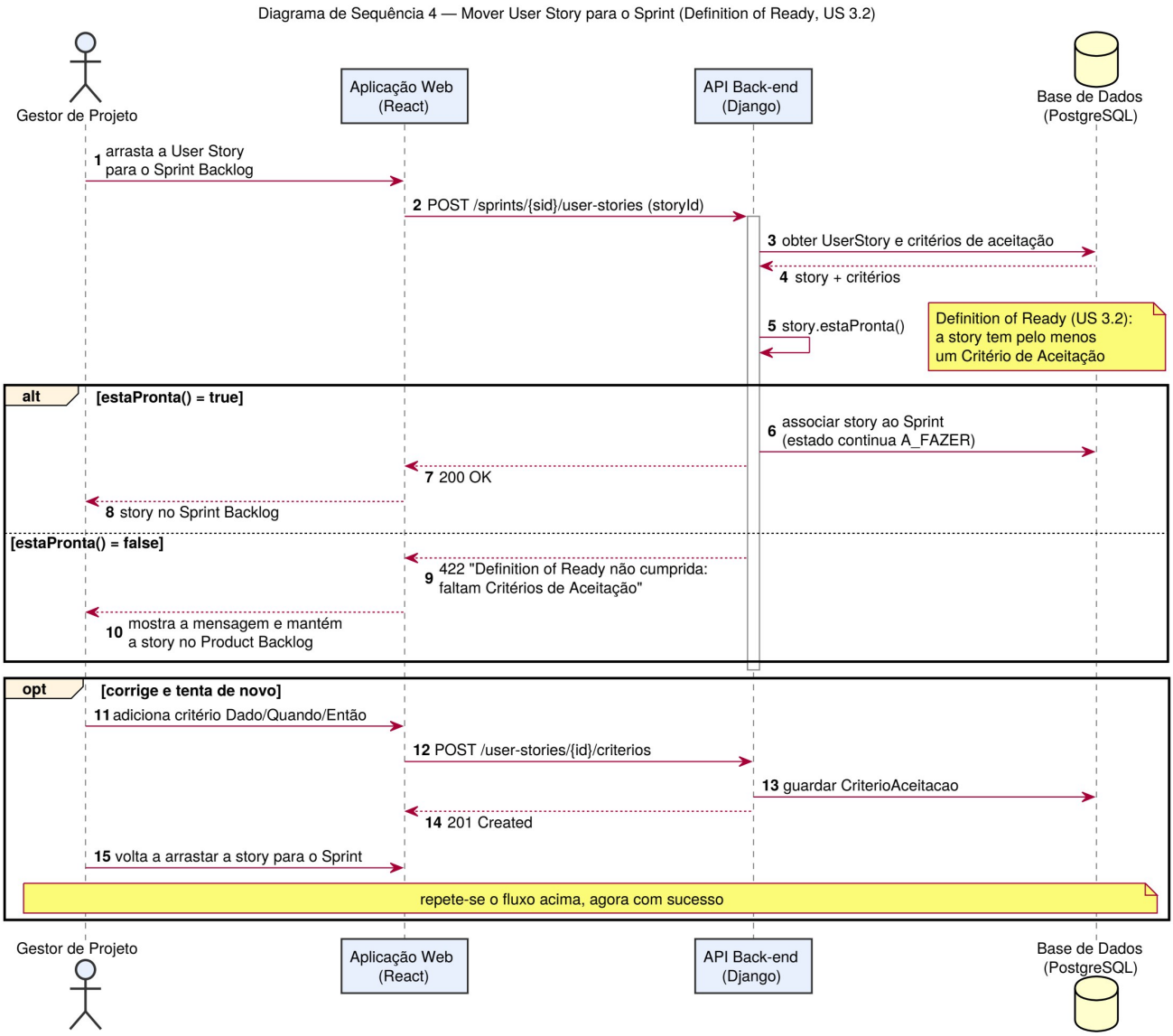


Diagrama de Sequência 4 — Definition of Ready — no PDF podes fazer zoom: a imagem tem resolução alta.

No código do Mini Sprint 3

Sequência 1: POST /api/v1/pedidos com 400/201 → `SubmeterPedidoSerializer`; depois envia o e-mail de confirmação e conclui a triagem sem IA (`pedidos/services.py`). Fila, Worker e IA entram no Sprint 4; fusão no Sprint 5.
Sequência 2 (aprovação): POST /pedidos/{id}/aprovar devolve 400 sem avaliação. Gerar User Story entra no Sprint 4.
Sequência 3: POST insere o voto (201 Created; 200 se já existia) e DELETE retira-o (200 OK); 401 sem token; 403 se o pedido estiver REJEITADO ou LANCADO. A aplicação envia POST ou DELETE consoante já tenha votado.
Sequência 4 entra no Sprint 4.

Perguntas que te podem fazer

P: Qual a diferença entre alt, opt e loop?

R: alt = if/else (um ramo de vários); opt = if sem else (pode ou não acontecer); loop = repetição enquanto a condição for verdadeira (as 2 tentativas à IA).

P: Porque é que a API responde 201 antes da IA terminar?

R: Porque a triagem vai para a fila e é feita pelo Worker. Assim o cliente nunca espera nem falha por causa da IA (ADR-004, RNF-07).

P: Porque 422 e não 400 no Diagrama 4?

R: O pedido está bem formado (400 seria dados inválidos); o que falha é uma regra de negócio sobre o estado da story, por isso 422 Unprocessable Entity.

P: Porque é que os participantes são esses?

R: São os contentores do C4 nível 2; usar os mesmos garante coerência entre a arquitetura (estrutura) e o comportamento.

9. Diagrama de Atividade — do Pedido à User Story

Junta num só fluxo o que os Diagramas de Sequência 1 e 2 mostram em mensagens, mas do ponto de vista do **processo**: quem faz cada passo e por que ordem. Cobre as US 2.1, 1.2, 1.5, 1.6 e 1.3.

Como está montado

- **Quatro raias:** Cliente, Sistema, Serviço de IA e Gestor de Produto. A raia diz quem é responsável.
- **repeat … repeat while:** o Cliente volta a preencher enquanto os dados forem inválidos (o texto é mantido).
- **fork / end fork (barras pretas):** o e-mail de confirmação e o pedido à IA correm **em paralelo** no Worker.
- **Decisões (losangos):** IA disponível? · É duplicado? · Avaliação completa? · Decisão (rejeitar/aprovar) · Esboço válido? · Rever o esboço (confirmar/descartar).
- **Vários fins:** pedido fundido, pedido rejeitado, story descartada ou story no Product Backlog.

Como ler: começa no círculo preto da raia do Cliente e segue as setas. Quando a seta muda de raia, muda quem faz o trabalho (ex.: "Submeter pedido" no Cliente → "Validar campos e anexo" no Sistema).

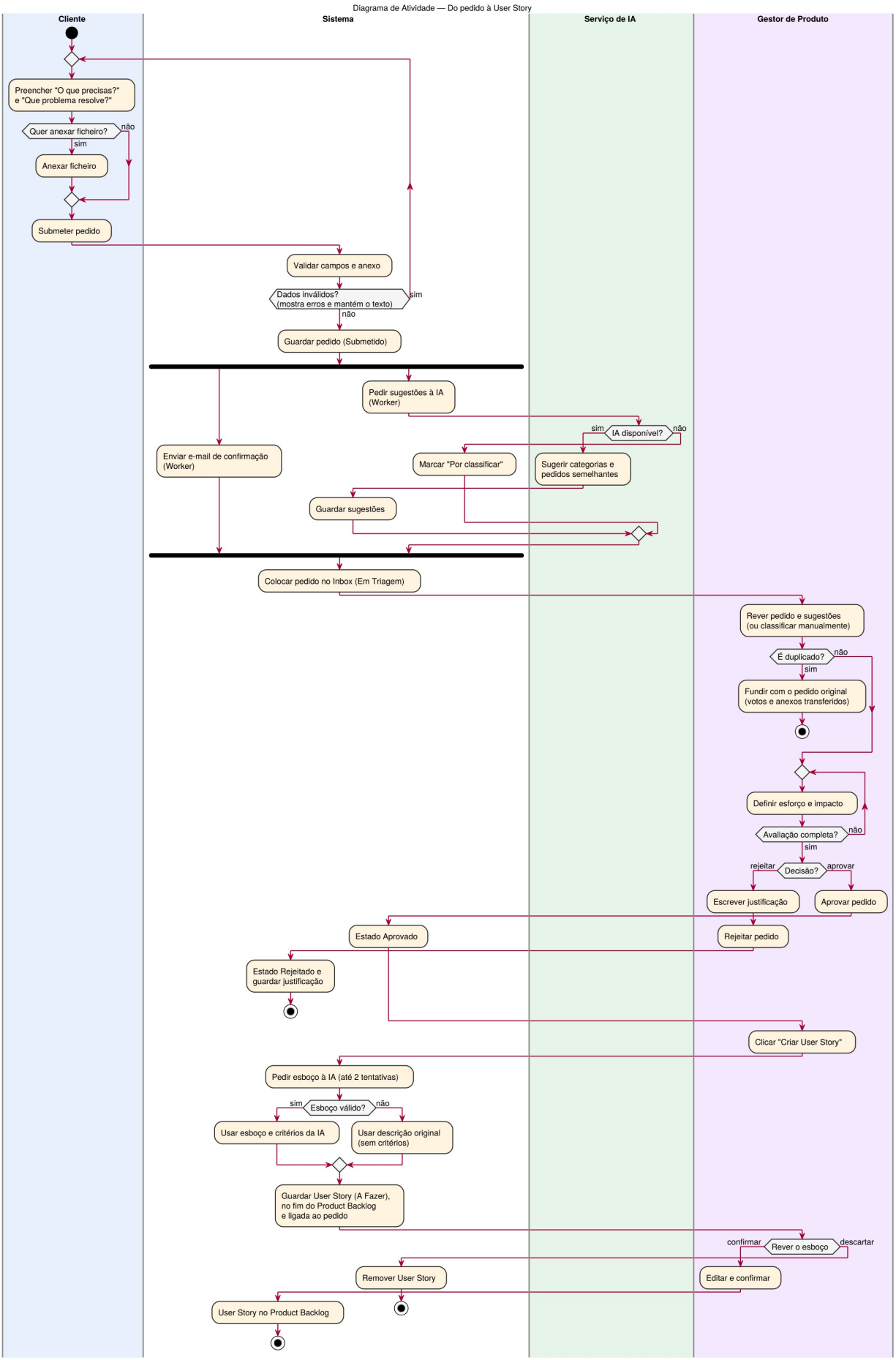


Diagrama de Atividade — Do pedido à User Story — no PDF podes fazer zoom: a imagem tem resolução alta.

Perguntas que te podem fazer

P: Qual a diferença entre o diagrama de atividade e o de sequência?

> **R:** O de sequência mostra mensagens entre participantes num cenário (técnico, com endpoints). O de atividade mostra o processo de negócio e quem é responsável por cada passo, com decisões e paralelismo.

P: O que representa a barra preta?

> **R:** Um fork (divide em ramos paralelos) ou um join (espera que todos os ramos terminem). Aqui: e-mail e IA ao mesmo tempo.

10. Coerência entre diagramas e código

Matriz de rastreabilidade

Conceito	Caso de uso	Classes	Estados	Sequência	Código (Mini Sprint 3)
Submeter pedido	Submeter pedido (Cliente)	Pedido	→ Submetido	Seq 1 (1–9)	POST /pedidos
Votar	Votar / Retirar voto	Voto, totalVotos()	votação fechada em Rejeitado / Lançado	Seq 3	POST/DELETE …/votos · services.votar (403 se fechado)
Avaliar	Avaliar esforço e impacto	esforco, impacto	guarda de aprovar()	Seq 2	PATCH /pedidos/{id}
Aprovar	Aprovar pedido «include» Avaliar	aprovar()	Em Triagem → Aprovado	Seq 2 (1–10)	Pedido.aprovar() · /aprovar
Rejeitar	Rejeitar pedido (com justificação)	rejeitar(j)	→ Rejeitado	Seq 1 (35–38)	Pedido.rejeitar() · /rejeitar
Perfis	Generalização de Utilizador	Perfil	—	—	contas/permissions.py
Fundir duplicado	Fundir pedidos «extend»	fundirCom()	→ fim	Seq 1 (29–34)	Sprint 5
Gerar User Story	Gerar User Story «extend» Aprovar	gerarUserStory()	→ A Fazer	Seq 2 (11–22)	Sprint 4
Definition of Ready	Mover US «include» Validar DoR	estaPronta()	guarda de iniciar()	Seq 4	Sprint 4

Diferenças que deves conhecer (e saber justificar)

Na apresentação é melhor seres tu a mostrá-las do que o professor encontrá-las:

Diagrama diz	Código faz	Porquê / o que fazer
ER: voto com PK composta (utilizador_id, pedido_id)	**Corrigido:** PK composta	`CompositePrimaryKey("utilizador", "pedido")` em `priorizacao/models.py`; o Voto deixa de aparecer no admin do Django (não suporta PK compostas).
Estados: aprovar() e rejeitar() só a partir de Em Triagem	**Corrigido**	A submissão conclui logo a triagem sem IA (Submetido → Em Triagem); com o Worker (Sprint 4) passa a ser assíncrona.
Seq 3: votar num pedido REJEITADO ou LANCADO dá 403	**Corrigido:** responde 403 e o botão fica desativado	Regra em `Pedido.aceita_votos`, testada em `test_pedido_terminado_nao_aceita_alteracoes_de_votos`.
Seq 3: POST vota (201), DELETE retira (200)	**Corrigido**	Clicar de novo no botão envia DELETE, por isso "votar de novo retira o voto" (guião 5.3) continua a valer.
EDS-12: Submeter pedido «include» Enviar confirmação por e-mail	**Corrigido:** envia o e-mail	Em desenvolvimento aparece na consola do backend; com o Worker (Sprint 4) passa para a fila.

Diagrama diz	Código faz	Porquê / o que fazer
EDS-12: Publicar Release (`Release.publicar()`)	**Corrigido no EDS-13**	A classe Release passou a ter `publicar(): void`, como pede o EDS-12.
Seq 1 e Atividade: Worker, IA, fusão	Ainda não existem	Worker e IA no Sprint 4; fusão de duplicados no Sprint 5.
EDS-12: Cliente tem "Explorar ideias em destaque"	Lista ordenada por votos + Dashboard "Mais votados"	Cumprido em parte; não há uma vista própria de "destaque".

11. Revisão rápida

Os números e nomes que convém saber de cor

- **5 diagramas de casos de uso** (comum + 1 por persona), **2 atores secundários** (Serviço de IA e GitHub) e 3 relações: associação, «include», «extend» (+ generalização entre atores).
- **14 classes e 3 enumerações** no diagrama de classes.
- **7 estados do Pedido:** SUBMETIDO, EM_TRIAGEM, APROVADO, REJEITADO, PLANEADO, EM_DESENVOLVIMENTO, LANCADO.
- **4 estados da User Story:** A_FAZER, EM_PROGRESSO, EM_REVISAO, CONCLUIDO.
- **4 perfis:** CLIENTE, GESTOR_PRODUTO, PROGRAMADOR, GESTOR_PROJETO (personas Sofia, Tiago, Rui, Inês).
- **3 tabelas de associação:** equipa_membro, pedido_tag, dependencia.
- **5 contentores:** SPA, API, Worker, Fila (Redis), Base de Dados — e **3 sistemas externos:** IA, GitHub, E-mail.
- **8 ADRs** (001–007 no EDS-10, 008 no Mini Sprint 3).
- **Códigos HTTP usados:** 200 OK, 201 Created, 204 No Content, 400 dados inválidos, 401 sem login, 403 sem permissão, 422 regra de negócio.

Exercício para treinar

Escolhe uma regra (por exemplo, "um voto por utilizador e pedido") e explica-a em voz alta passando por todos os diagramas: User Story que a pede (US 2.2) → caso de uso Votar em pedido (EDS-12) → classe Voto e a nota → PK composta no ER → Diagrama de Sequência 3 → teste `test_base_de_dados_impede_voto_repetido`. Repete com "aprovar só com esforço e impacto" e "rejeitar exige justificação". Se conseguires fazer isto para as três, dominas a ligação entre os diagramas.

Onde está cada coisa

Ficheiro	Conteúdo
docs/EDS-12_Casos_de_Uso.pdf	Casos de uso: comuns + 1 diagrama por persona
docs/EDS-10_Estilo_Arquitetural_ADRs.pdf	Estilo arquitetural, ADR-001 a 007, C4 nível 2 e 3
docs/EDS-11_Diagramas_C4.pdf	C4 nível 1, 2 e 3 (com o código PlantUML)
docs/EDS-13_Diagrama_de_Classes.pdf	Diagrama de classes e 15 decisões de design
docs/EDS-14_Modelo_ER.pdf	Modelo entidade-relação
docs/Diagramas_de_Estados.pdf	Estados do Pedido e da User Story
docs/Diagrama1…4_*.pdf	Diagramas de sequência 1 a 4
docs/Diagrama_Atividade.pdf	Diagrama de atividade
docs/adr/ADR-008-ferramenta-ia-programacao.md	ADR-008
docs/Mini_Sprint3_Execucao.md	Ligação backlog → código e decisões de implementação